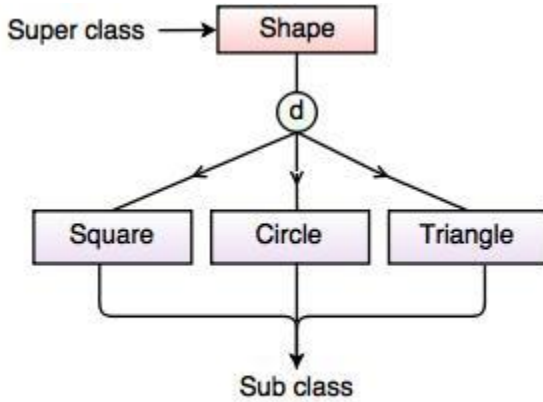


Contents

CHAPTER ONE	1
1. Introduction to Database Systems.....	1
1.1 Introduction	1
1.2 Data Handling Approach.....	2
1.2.1 Manual Data Handling approach	2
1.2.2 Traditional File Based Data Handling approach.....	2
1.2.3 Database Data Handling approach.....	4
1.3 Users and actors of Database system	5
1.3.1 Database Designer	6
1.3.2 Database Administrator	6
1.3.3 Application Developers	6
1.3.4 End-Users.....	7
1.4 Database Management System.....	7
1.4.1 Components and Interfaces of DBMS	7
1.4.2 Functions of DBMS	9
CHAPTER TWO	9
2 Database System Architecture	9
2.1 Database Architecture	9
2.1.1 Types of DBMS Architecture	9
2.2 Data models and conceptual models	11
2.2.1 Data models and conceptual models.....	11
2.3 Database Languages.....	15

2.3.1	The Data Definition Language (DDL).....	15
2.3.2	The Data Manipulation Language (DML).....	16
2.3.3	Transaction Control language (TCL).....	16
2.3.4	Data Control Language (DCL)	16
CHAPTER THREE		16
3	Database Modeling	16
3.1	Database Development Life Cycle.....	16
3.2	The Entity Relationship (ER) Model	17
3.2.1	Entity.....	17
3.2.2	Attributes.....	17
3.2.3	Entity-Set and Keys	18
3.2.4	Relationship	18
3.2.5	Relationship Set	18
3.2.6	Degree of Relationship	18
3.2.7	Mapping Cardinalities.....	18
3.3	ER Diagram Representation.....	20
3.3.1	Entity.....	20
3.3.2	Attributes.....	20
3.3.3	Relationship	22
3.4	Mapping ER-models to relational tables.....	24
3.4.1	Mapping Entity	24
3.4.2	Mapping Relationship.....	24
3.4.3	Mapping Weak Entity Sets	25

3.4.4	Mapping Hierarchical Entities	25
3.5	Enhanced Entity Relationship (EER) Model	26
3.5.1	Sub Class and Super Class	26
		
	Fig. Super class/Sub class Relationship	27
3.5.2	Specialization and Generalization.....	27
3.5.3	Category or Union.....	28
3.5.4	Aggregation.....	28
3.6	The Relational Database Model	29
3.6.1	Concepts.....	29
3.6.2	Constraints	29
3.6.3	Key Constraints.....	30
3.6.4	Domain Constraints	30
3.6.5	Referential integrity Constraints	30
CHAPTER FOUR.....		30
4	Functional Dependency and Normalization.....	30
4.1	Functional Dependency	30
4.2	Normalization.....	31

4.2.1	First normal form (1NF)	31
4.2.2	Second normal form (2NF)	31
4.2.3	Third normal form (3NF)	32
4.3	Conclusion	34
CHAPTER FIVE		34
5	Record Storage and Primary File Organization	34
5.1	File Storage System	34
5.1.1	Memory Hierarchy	35
5.1.2	Magnetic Disks	35
5.1.3	Redundant Array of Independent Disks	35
5.2	File Structure	37
5.2.1	File Organization	38
5.2.2	File Operations	38
5.2.3	Indexing	39
CHAPTER SIX		42
6	The Relational Algebra and Relational Calculus	42
6.1	Relational Algebra	42
6.1.1	Fundamental Operations of Relational Algebra	43
6.1.2	Additional Operations	45
6.1.3	Extended Operations	46
6.2	Introduction to Relational Calculus	47
6.2.1	Domain Relational Calculus	47
CHAPTER SEVEN		48

7	Structured Query Language (SQL)	48
7.1	Introduction	48
7.1.1	SQL Data Definition Language (DDL).....	48
7.1.2	SQL Data Manipulation Language (DML)	48
7.2	Schema Definition in SQL	49
7.2.1	Schema Creation and Modification.....	49
7.2.2	Table Creation and Modification	49
7.2.3	Index Creation and Modification	52
7.2.4	INSERT, UPDATE and DELETE	54
7.2.5	Nested Subqueries and Complex Queries	56

CHAPTER ONE

1. Introduction to Database Systems

1.1 Introduction

An organization must have accurate and reliable data for effective decision making. To this end, the organization maintains records on the various facets maintaining relationships among them. Such related data are called a database. A database system is an integrated collection of related files, along with details of the interpretation of the data contained therein. Basically, database system is nothing more than a computer-based record keeping system i.e., a system whose overall purpose is to record and maintain information/data. A database management system (DBMS) is a software system that allows access to data contained in a database. The objective of the DBMS is to provide a convenient and effective method of defining, storing and retrieving the information contained in the database.

Generally, a database is an organized collection of related information. The organized information or database serves as a base from which desired information can be retrieved or decision made by further recognizing or processing the data. People use several databases in their day-to-day life. Dictionary, Telephone directory, Library catalog, etc are example for databases where the entries are arranged according to alphabetical or classified order.

The term 'DATA' can be defined as the value of an attribute of an entity. Any collection of related data items of entities having the same attributes may be referred to as a 'DATABASE'. Mere collection of data does not make it a database; the way it is organized for effective and efficient use makes it a database.

The technology that emerged to process data of various kinds is grossly termed as 'DATABASE MANAGEMENT TECHNOLOGY' and the resulting software are known as 'DATABASE MANAGEMENT SYSTEM' (DBMS) which they manage a computer stored database or collection of data.

An entity set is a set of entities of the same type that share the same properties or attributes. An entity is represented by set of attributes. An attribute is also referred as data item, data element, data field, etc. Attributes are descriptive properties possessed by each member of an entity set. A grouping of related entities becomes an entity set.

For example: In a library environment,

Entity Set: Catalogue

Entity: Books, Journals, AV-Materials, etc

Attributes: contains ISBN, title, author, or publisher, etc.

The word 'DATA' means a fact or more specially a value of attribute of an entity. An entity in general, may be an object, idea, event, condition or situation. A set of attributes describes an entity. Information in a form which can be processed by a raw computer is called data. Data are raw material of information. The term 'BASE' means the support, foundation or key ingredient of anything. Therefore, base supports data. A 'DATABASE' can be conceived as a system whose base, whose key concept, is simply a particular way of handling data. In other words, a database is nothing more than a computer-based record keeping.

The objective of database is to record and maintain information. The primary function of the database is the service and support of information system which satisfies cost. In short, —A database is an organized collection of related information stored with minimum redundancy, in a manner that makes them accessible for multiple applications”.

1.2 Data Handling Approach

1.2.1 Manual Data Handling approach

In the manual approach, data storage and retrieval follow the primitive and traditional way of information handling where cards and paper are used for the purpose. The data storage and retrieval will be performed using human labor.

- ✓ Files for as many event and objects as the organization has are used to store information.
- ✓ Each of the files containing various kinds of information is labelled and stored in one or more cabinets.
- ✓ The cabinets could be kept in safe places for security purpose based on the sensitivity of the information contained in it.
- ✓ Insertion and retrieval are done by searching first for the right cabinet then for the right the file then the information.
- ✓ One could have an indexing system to facilitate access to the data

Limitations of the Manual approach

- ✓ Prone to error
- ✓ Difficult to update, retrieve, integrate
- ✓ You have the data but it is difficult to compile the information
- ✓ Limited to small size information
- ✓ Cross referencing is difficult

An alternative approach of data handling is a computerized way of dealing with the information. The computerized approach could also be either decentralized or centralized base on where the data resides in the system.

1.2.2 Traditional File Based Data Handling approach

After the introduction of Computer for data processing to the business community, the need to use the device for data storage and processing increase. There were, and still are, several computer

applications with file-based processing used for the purpose of data handling. Even though the approach evolved over time, the basic structure is still similar if not identical.

- ✓ File based systems were an early attempt to computerize the manual filing system.
- ✓ This approach is the decentralized computerized data handling method.
- ✓ A collection of application programs performs services for the end-users. In such systems, every application program that provides service to end users define and manage its own data
- ✓ Such systems have number of programs for each of the different applications in the organization.
- ✓ Since every application defines and manages its own data, the system is subjected to serious data duplication problem.
- ✓ File, in traditional file-based approach, is a collection of records which contains logically related data

Limitations of the Traditional File Based approach

As business application become more complex demanding more flexible and reliable data handling methods, the shortcomings of the file-based system became evident. These shortcomings include, but not limited to:

- ✓ Separation or Isolation of Data: Available information in one application may not be known. Data Synchronization is done manually.
- ✓ Limited data sharing- every application maintains its own data.
- ✓ Lengthy development and maintenance time
- ✓ Duplication or redundancy of data (money and time cost and loss of data integrity)
- ✓ Data dependency on the application- data structure is embedded in the application; hence, a change in the data structure needs to change the application as well.
- ✓ Incompatible file formats or data structures (e.g. —Cll and COBOL) between different applications and programs creating inconsistency and difficulty to process jointly.
- ✓ Fixed query processing which is defined during application development

The limitations for the traditional file-based data handling approach arise from two basic reasons.

1. Definition of the data is embedded in the application program which makes it difficult to modify the database definition easily.
2. No control over the access and manipulation of the data beyond that imposed by the application programs.

The most significant problem experienced by the traditional file-based approach of data handling can be formalized by what is called —update anomaliesll. We have three types of update anomalies;

1. **Modification Anomalies:** a problem experienced when one ore more data value is modified on one application program but not on others containing the same data set.

2. **Deletion Anomalies:** a problem encountered where one record set is deleted from one application but remain untouched in other application programs.
3. **Insertion Anomalies:** a problem experienced whenever there is new data item to be recorded, and the recording is not made in all the applications. And when same data item is inserted at different applications, there could be errors in encoding which makes the new data item to be considered as a totally different object.

1.2.3 Database Data Handling approach

Following a famous paper written by Dr. Edgar Frank Codd in 1970, database systems changed significantly. Codd proposed that database systems should present the user with a view of data organized as tables called relations. Behind the scenes, there might be a complex data structure that allowed rapid response to a variety of queries. But, unlike the user of earlier database systems, the user of a relational system would not be concerned with the storage structure. Queries could be expressed in a very high-level language, which greatly increased the efficiency of database programmers. The database approach emphasizes the integration and sharing of data throughout the organization.

Thus, in Database Approach:

- ✓ Database is just a computerized record keeping system or a kind of electronic filing cabinet.
- ✓ Database is a repository for collection of computerized data files.
- ✓ Database is a shared collection of logically related data and description of data designed to meet the information needs of an organization. Since it is a shared corporate resource, the database is integrated with minimum amount of or no duplication.
- ✓ Database is a collection of logically related data where these logically related data comprise entities, attributes, relationships, and business rules of an organization's information
- ✓ In addition to containing data required by an organization, database also contains a description of the data which is known as Metadata or Data Dictionary or Systems Catalogue or Data about Data or sometimes Data Directory.
- ✓ Since a database contains information about the data (metadata), it is called a self-descriptive collection of integrated records.
- ✓ The purpose of a database is to store information and to allow users to retrieve and update that information on demand.
- ✓ Database is designed once and used simultaneously by many users.
- ✓ Unlike the traditional file-based approach in database approach there is program data independence. That is the separation of the data definition from the application. Thus, the application is not affected by changes made in the data structure and file organization.
- ✓ Each database application will perform the combination of: Creating database, Reading, Updating and Deleting data.

Benefits of the database approach

- **Data can be shared:** two or more users can access and use same data instead of storing data in redundant manner for each user.
- **Improved accessibility of data:** by using structured query languages, the users can easily access data without programming experience.
- **Redundancy can be reduced:** isolated data is integrated in database to decrease the redundant data stored at different applications.
- **Quality data can be maintained:** the different integrity constraints in the database approach will maintain the quality leading to better decision making
- **Inconsistency can be avoided:** controlled data redundancy will avoid inconsistency of the data in the database to some extent.
- **Transaction support can be provided:** basic demands of any transaction support systems are implanted in a full-scale DBMS.
- **Integrity can be maintained:** data at different applications will be integrated together with additional constraints to facilitate validity and consistency of shared data resource.
- **Security measures can be enforced:** the shared data can be secured by having different levels of clearance and other data security mechanisms.
- **Improved decision support:** the database will provide information useful for decision making.
- **Standards can be enforced:** the different ways of using and dealing with data by different unite of an organization can be balanced and standardized by using database approach.
- **Compactness:** since it is an electronic data handling method, the data is stored compactly (no voluminous papers)
- **Speed:** data storage and retrieval is fast as it will be using the modern fast computer systems.
- **Less labor:** unlike the other data handling methods, data maintenance will not demand much resource.
- **Centralized information control:** since relevant data in the organization will be stored at one repository, it can be controlled and managed at the central level.

Limitations and risk of Database Approach

- ✓ Introduction of new professional and specialized personnel
- ✓ Complexity in designing and managing data
- ✓ The cost and risk during conversion from the old to the new system
- ✓ High cost to be incurred to develop and maintain the system
- ✓ Complex backup and recovery services from the user's perspective
- ✓ Reduced performance due to centralization and data independency
- ✓ High impact on the system when failure occurs to the central system

1.3 Users and actors of Database system

As people are one of the components in DBMS environment, there are group of roles played by different stakeholders of the designing and operation of a database system.

1.3.1 Database Designer

Database designers are responsible for identifying the data to be stored in the database and for choosing appropriate structures to represent and store this data. These tasks are mostly undertaken before the database is actually implemented and populated with data. It is the responsibility of database designers to communicate with all prospective database users in order to understand their requirements and to create a design that meets these requirements. In many cases, the designers are on the staff of the DBA and may be assigned other staff responsibilities after the database design is completed.

Database designers typically interact with each potential group of users and develop views of the database that meet the data and processing requirements of these groups. Each view is then analyzed and integrated with the views of other user groups. The final database design must be capable of supporting the requirements of all user groups.

1.3.2 Database Administrator

- ✓ Responsible to oversee, control and manage the database resources (the database itself, the DBMS and other related software)
- ✓ Authorizing access to the database
- ✓ Coordinating and monitoring the use of the database
- ✓ Responsible for determining and acquiring hardware and software resources
- ✓ Accountable for problems like poor security, poor performance of the system
- ✓ Involves in all steps of database development
- ✓ We can have further classifications of this role in big organizations having huge amount of data and user requirement.
 - **Data Administrator (DA):** is responsible on management of data resources. This involves in database planning, development, maintenance of standards policies and procedures at the conceptual and logical design phases.
 - **Database Administrator (DBA):** This is more technically oriented role. DBA is responsible for the physical realization of the database. It is involved in physical design, implementation, security and integrity control of the database.

1.3.3 Application Developers

Once the database has been implemented, the application programs that provide the required functionality for the end-users must be implemented. This is the responsibility of the application developers. Typically, the application developers work from a specification produced by systems analysts. Each program contains statements that request the DBMS to perform some operation on the database, which includes retrieving data, inserting, updating, and deleting data. The programs may be written in a third-generation or fourth-generation programming language, as discussed previously.

1.3.4 End-Users

End users are the people whose jobs require access to the database for querying, updating, and generating reports; the database primarily exists for their use. There are several categories of end users:

- ✓ **Casual end users** occasionally access the database, but they may need different information each time. They use a sophisticated database query interface to specify their requests and are typically middle- or high-level managers or other occasional browsers.
- ✓ **Naive or parametric end users** make up a sizable portion of database end users. Their main job function revolves around constantly querying and updating the database, using standard types of queries and updates—called canned transactions—that have been carefully programmed and tested. Many of these tasks are now available as mobile apps for use with mobile devices. The tasks that such users perform are varied. A few examples are:
 - Bank customers and tellers check account balances and post withdrawals and deposits.
 - Reservation agents or customers for airlines, hotels, and car rental companies check availability for a given request and make reservations.
 - Employees at receiving stations for shipping companies enter package identifications via bar codes and descriptive information through buttons to update a central database of received and in-transit packages.
 - Social media users post and read items on social media Web sites.
- ✓ **Sophisticated end users** include engineers, scientists, business analysts, and others who thoroughly familiarize themselves with the facilities of the DBMS in order to implement their own applications to meet their complex requirements.
- ✓ **Standalone users** maintain personal databases by using ready-made program packages that provide easy-to-use menu-based or graphics-based interfaces. An example is the user of a financial software package that stores a variety of personal financial data.

Standalone users typically become very proficient in using a specific software package.

1.4 Database Management System

Database Management System (DBMS) is a Software package used for providing EFFICIENT, CONVENIENT and SAFE MULTI-USER (many people/programs accessing same database, or even same data, simultaneously) storage of and access to MASSIVE amounts of PERSISTENT (data outlives programs that operate on it) data. A DBMS also provides a systematic method for creating, updating, storing, retrieving data in a database. DBMS also provides the service of controlling data access, enforcing data integrity, managing concurrency control, and recovery.

1.4.1 Components and Interfaces of DBMS

We can identify five major components in the DBMS environment: hardware, software, data, procedures, and people.

Hardware: The DBMS and the applications require hardware to run. The hardware can range from a single personal computer to a single mainframe or a network of computers.

The particular hardware depends on the organization 's requirements and the DBMS used. Some DBMSs run only on particular hardware or operating systems, while others run on a wide variety of hardware and operating systems. A DBMS requires a minimum amount of main memory and disk space to run, but this minimum configuration may not necessarily give acceptable performance.

Software: The software component comprises the DBMS software itself and the application programs, together with the operating system, including network software if the DBMS is being used over a network. Typically, application programs are written in a third-generation programming language (3GL), such as C, C++, C#, Java, Visual Basic, COBOL, Fortran, Ada, or Pascal, or a fourth-generation language (4GL), such as SQL, embedded in a third-generation language.

Data: Perhaps the most important component of the DBMS environment—certainly from the end-users 'point of view—is the data. I observe that the data acts as a bridge between the machine components and the human components. The database contains both the operational data and the metadata, the —data about data. || The structure of the database is called the schema.

Procedures refer to the instructions and rules that govern the design and use of the database. The users of the system and the staff who manage the database require documented procedures on how to use or run the system. These may consist of instructions on how to:

- ✓ Log on to the DBMS.
- ✓ Use a particular DBMS facility or application program.
- ✓ Start and stop the DBMS.
- ✓ Make backup copies of the database.
- ✓ Handle hardware or software failures. This may include procedures on how to identify the failed component, how to fix the failed component (for example, telephone the appropriate hardware engineer), and, following the repair of the fault, how to recover the database.
- ✓ Change the structure of a table, reorganize the database across multiple disks, improve performance, or archive data to secondary storage.

People: The final component is the people involved with the system we can identify four distinct types of people who participate in the DBMS environment: data and database administrators, database designers, application developers, and end-users. This component is composed of the people in the organization that are responsible or play a role in designing, implementing, managing, administering and using the resources in the database.

1.4.2 Functions of DBMS

In this section we look at the types of function and service we would expect a DBMS to provide lists different services that should be provided by any full-scale DBMS

- A. Data storage, retrieval, and update:** DBMS must furnish users with the ability to store, retrieve, and update data in the database.
- B. A user-accessible catalog:** A DBMS must furnish a catalog in which descriptions of data items are stored and which is accessible to users. A key feature of the ANSI-SPARC architecture is the recognition of an integrated system catalog to hold data about the schemas, users, applications, and so on. The catalog is expected to be accessible to users as well as to the DBMS.
- C. Transaction support:** A DBMS must furnish a mechanism which will ensure either that all the updates corresponding to a given transaction are made or that none of them is made. A transaction is a series of actions, carried out by a single user or application program, which accesses or changes the contents of the database.
- D. Concurrency control services:** A DBMS must furnish a mechanism to ensure that the database is updated correctly when multiple users are updating the database concurrently. One major objective in using a DBMS is to enable many users to access shared data concurrently. Concurrent access is relatively easy if all users are only reading data, as there is no way that they can interfere with one another. However, when two or more users are accessing the database simultaneously and at least one of them is updating data, there may be interference that can result in inconsistencies. The DBMS must ensure that, when multiple users are accessing the database, interference cannot occur.
 - ✓ **Recovery services:** A DBMS must furnish a mechanism for recovering the database in the event that the database is damaged in any way.
 - ✓ **Authorization services:** A DBMS must furnish a mechanism to ensure that only authorized users can access the database.

CHAPTER TWO

2 Database System Architecture

2.1 Database Architecture

A Database Architecture is a representation of DBMS design. It helps to design, develop, implement, and maintain the database management system. A DBMS architecture allows dividing the database system into individual components that can be independently modified, changed, replaced, and altered. It also helps to understand the components of a database. A Database stores critical information and helps access data quickly and securely. Therefore, selecting the correct Architecture of DBMS helps in easy and efficient data management.

2.1.1 Types of DBMS Architecture

There are mainly three types of DBMS architecture:

- ✓ One Tier Architecture (Single Tier Architecture)
- ✓ Two Tier Architecture
- ✓ Three Tier Architecture

2.1.1.1 1-Tier Architecture

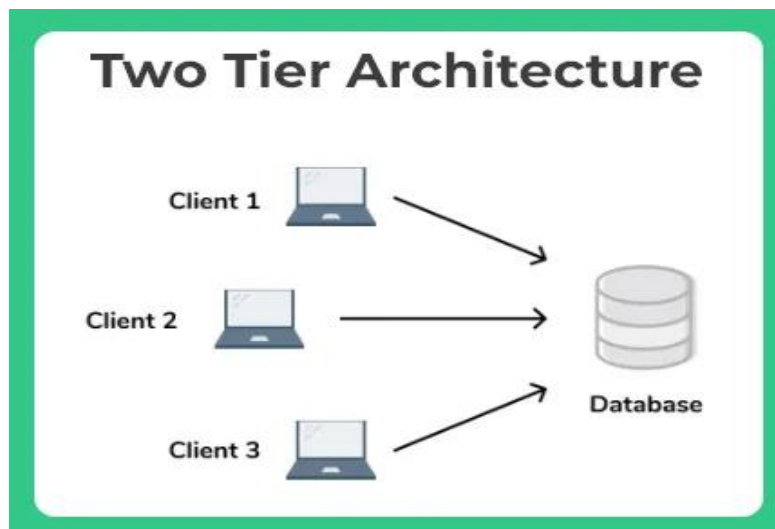
1 Tier Architecture in DBMS is the simplest architecture of Database in which the client, server, and Database all reside on the same machine. A simple one tier architecture example would be anytime you install a Database in your system and access it to practice SQL queries. But such architecture is rarely used in production.



Single Tier Architecture

2.1.1.2 2-Tier Architecture

A **2 Tier Architecture** in DBMS is a Database architecture where the presentation layer runs on a client (PC, Mobile, Tablet, etc.), and data is stored on a server called the second tier. Two tier architecture provides added security to the DBMS as it is not exposed to the end-user directly. It also provides direct and faster communication.



In the above 2 Tier client-server architecture of database management system, we can see that one server is connected with clients 1, 2, and 3.

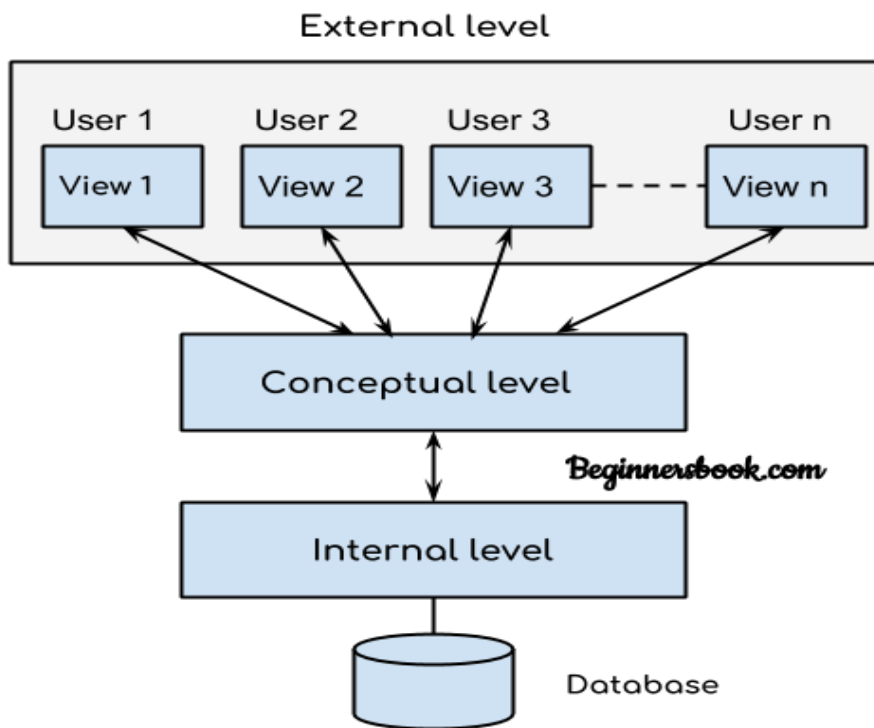
Two Tier Architecture Example: A Contact Management System created using MS- Access.

2.1.1.3 3-Tier Architecture

A **3 Tier Architecture** in DBMS is the most popular client server architecture in DBMS in which the development and maintenance of functional processes, logic, data access, data storage, and user interface is done independently as separate modules. Three Tier architecture contains a presentation layer, an application layer, and a database server.

3-Tier database Architecture design is an extension of the 2-tier client-server architecture. A 3-tier architecture has the following layers:

1. Presentation layer (your PC, Tablet, Mobile, etc.)
2. Application layer (server)
3. Database Server



2.2 Data models and conceptual models

2.2.1 Data models and conceptual models

A specific DBMS has its own specific Data Definition Language to define a database schema, but this type of language is too low level to describe the data requirements of an organization in a way that is readily understandable by a variety of users. We need a higher-level language. Such a higher-level description of the database schema is called *data-model*.

2.2.1.1 Data Model

A set of concepts to describe the *structure* of a database, and certain *constraints* that the database should obey.

A **data model** is a description of the way that data is stored in a database. Data model helps to understand the relationship between entities and to create the most effective structure to hold data.

Data Model is a collection of tools or concepts for describing

- ✓ Data
- ✓ Data relationships
- ✓ Data semantics
- ✓ Data constraints

The main **purpose** of Data Model is to represent the data in an understandable way. **Categories of data models include:**

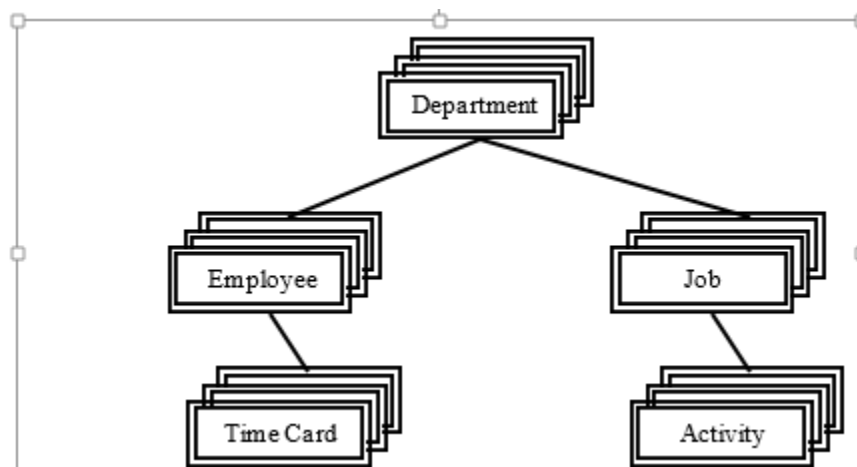
1. Object-based
2. Record-based
3. Physical

1. **Record-based Data Models:** Consist of a number of fixed format records. Each record type defines a fixed number of fields; each field is typically of a fixed length.

- ✓ Hierarchical Data Model
- ✓ Network Data Model
- ✓ Relational Data Model

Hierarchical Model

- ✓ The simplest data model
- ✓ Record type is referred to as node or segment
- ✓ The top node is the root node
- ✓ Nodes are arranged in a hierarchical structure as sort of upside-down tree
- ✓ A parent node can have more than one child node
- ✓ A child node can only have one parent node
- ✓ The relationship between parent and child is one-to-many
- ✓ Relation is established by creating physical link between stored records (each is stored with a predefined access path to other records)
- ✓ To add new record type or relationship, the database must be redefined and then stored in a new form.



Advantages Of Hierarchical Data Model:

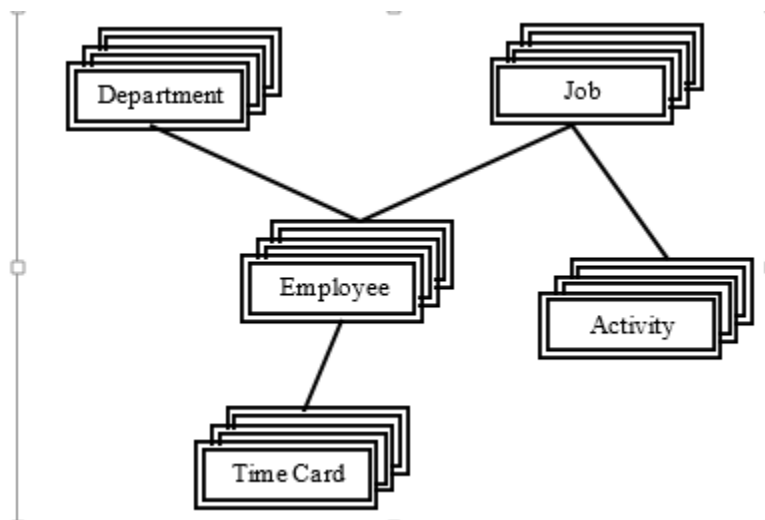
- ✓ Hierarchical Model is simple to construct and operate on
 - Corresponds to a number of natural hierarchically organized domains – e.g., assemblies in manufacturing, personnel organization in companies
 - Language is simple; uses constructs like GET, GET UNIQUE, GET NEXT, GET NEXT WITHIN PARENT etc.

Disadvantages Of Hierarchical Data Model:

- ✓ Navigational and procedural nature of processing
 - Database is visualized as a linear arrangement of records
 - Little scope for “query optimization”

Network Model

- ✓ Allows record types to have more than one parent unlike hierarchical model
- ✓ A network data models sees records as set members
- ✓ Each set has an owner and one or more members
- ✓ Allow no many to many relationships between entities
- ✓ Like hierarchical model network model is a collection of physically linked records.
- ✓ Allow member records to have more than one owner



Advantages Of Network Data Model:

- ✓ Network Model is able to model complex relationships and represents semantics of add/delete on the relationships.
 - Can handle most situations for modeling using record types and relationship types.
 - Language is navigational; uses constructs like FIND, FIND member, FIND owner, FIND NEXT within set, GET etc. Programmers can do optimal navigation through the database.

Disadvantages Of Network Data Model:

- ✓ Navigational and procedural nature of processing
 - Database contains a complex array of pointers that thread through a set of records.
 - Little scope for automated “query optimization”

Relational Data Model

- ✓ Developed by **Dr. Edgar Frank Codd in 1970** (famous paper, ‘A Relational Model for Large Shared Data Banks’)
- ✓ Terminologies originates from the branch of mathematics called set theory and predicate logic and is based on the mathematical concept called **Relation**
- ✓ Can define more flexible and complex relationship
- ✓ Viewed as a collection of tables called —Relations equivalent to collection of record types
- ✓ **Relation**: Two-dimensional table
- ✓ Stores information or data in the form of tables à **rows and columns**
- ✓ A **row** of the table is called tuple à equivalent to **record**
- ✓ A **column** of a table is called attribute à equivalent to **fields**
- ✓ Data value is the value of the Attribute
- ✓ Records are related by the data stored jointly in the fields of records in two tables or files. The related tables contain information that creates the relation • The tables seem to be independent but are related somehow.
- ✓ No physical consideration of the storage is required by the user
- ✓ Many tables are merged together to come up with a new virtual view of the relationship

2.2.1.2 Conceptual Data Model

Many data models have been proposed, which we can categorize according to the types of concepts they use to describe the database structure. High-level or **conceptual data models** provide concepts that are close to the way many users perceive data, whereas low-level or physical data models provide concepts that describe the details of how data is stored on the computer storage media, typically magnetic disks. Concepts provided by low-level data models are generally meant for computer specialists, not for end users. Between these two extremes is a class of representational (or implementation) data models, which provide concepts that may be easily understood by end users but that are not too far removed from the way data is organized in computer storage. Representational data models hide many details of data storage on disk but can be implemented on a computer system directly.

Conceptual data models use concepts such as entities, attributes, and relationships.

An entity represents a real-world object or concept, such as an employee or a project from the mini-world that is described in the database.

An attribute represents some property of interest that further describes an entity, such as the employee ‘s name or salary.

A relationship among two or more entities represents an association among the entities, for example, a works-on relationship between an employee and a project.

2.3 Database Languages

A database language consists of four parts:

- ✓ **A Data Definition Language (DDL)**
- ✓ **Data Manipulation Language (DML),**
- ✓ **Transaction control language (TCL) and**
- ✓ **Data control language (DCL).**

The DDL is used to specify the database schema and the DML is used to both read and update the database. These languages are called data sublanguages because they do not include constructs for all computing needs such as conditional or iterative statements, which are provided by the high-level programming languages. Many DBMSs have a facility for embedding the sublanguage in a high-level programming language such as COBOL, FORTRAN, Pascal, Ada, and `_C`, C++, Java, or Visual Basic. In this case, the high-level language is sometimes referred to as the host language. To compile the embedded file, the commands in the data sublanguage are first removed from the host language program and replaced by function calls. The pre-processed file is then compiled, placed in an object module, linked with a DBMS-specific library containing the replaced functions, and executed when required. Most data sublanguages also provide non-embedded, or interactive, commands that can be input directly from a terminal.

2.3.1 The Data Definition Language (DDL)

It is a language that allows the DBA or user to describe and name the entities, attributes, and relationships required for the application, together with any associated integrity and security constraints.

The database schema is specified by a set of definitions expressed by means of a special language called a Data Definition Language. The DDL is used to define a schema or to modify an existing one. It cannot be used to manipulate data.

The result of the compilation of the DDL statements is a set of tables stored in special files collectively called the system catalog. The system catalog integrates the metadata that is data that describes objects in the database and makes it easier for those objects to be accessed or manipulated. The metadata contains definitions of records, data items, and other objects that are of interest to users or are required by the DBMS. The DBMS normally consults the system catalog before the actual data is accessed in the database. The terms data dictionary and data directory are also used to describe the system catalog, although the term `_data dictionary` 'usually refers to a more general software system than a catalog for a DBMS.

Which defines the database structure or schema. Specifies additional properties or constraints of the data. The database system is checking these constraints every time the database is updated.

Example: CREATE: create object in database

ALTER: alter the structure of database

DROP: deletes object from database

RENAME: rename the object

2.3.2 The Data Manipulation Language (DML)

It is a language that provides a set of operations to support the basic data manipulation operations on the data held in the databases. Data manipulation operations usually include the following:

- insertion of new data into the database;
- modification of data stored in the database;
- retrieval of data contained in the database; ü Deletion of data from the database.

Therefore, one of the main functions of the DBMS is to support a data manipulation language in which the user can construct statements that will cause such data manipulation to occur. Data manipulation applies to the external, conceptual, and internal levels. However, at the internal level we must define rather complex low-level procedures that allow efficient data access. In contrast, at higher levels, emphasis is placed on ease of use and effort is directed at providing efficient user interaction with the system.

2.3.3 Transaction Control language (TCL)

It used to manage transaction in database and the change made by data manipulation language statements.

Transaction: the logical unit of work which consists of some operations to control some tasks.

Example: COMMIT: used to permanently save any transaction into the database.

ROLLBACK: restore the database to last committee state.

2.3.4 Data Control Language (DCL)

It used to control access to data stored in database (Authorization)

Example: GRANT: allows specified users to perform specified tasks. REVOKE: cancel pervious granted or denied permission

The part of a DML that involves data retrieval is called a query language. A query language can be defined as a high-level special-purpose language used to satisfy diverse requests for the retrieval of data held in the database. The term `_query` 'is therefore reserved to denote a retrieval statement expressed in a query language or specifies data to retrieve rather than how to retrieve it.

CHAPTER THREE

3 Database Modeling

3.1 Database Development Life Cycle

As it is one component in most information system development tasks, there are several steps in designing a database system. Here more emphasis is given to the design phases of the system development life cycle. The major steps in database design are;

1. **Planning:** that is identifying information gap in an organization and propose a database solution to solve the problem.
2. **Analysis:** that concentrates more on fact finding about the problem or the opportunity. Feasibility analysis, requirement determination and structuring, and selection of best design method are also performed at this phase.

3. **Design:** in database development more emphasis is given to this phase. The phase is further divided into three sub-phases.
 - A. **Conceptual Design:** concise description of the data, data type, relationship between data and constraints on the data.
 - ✓ There is no implementation or physical detail consideration.
 - ✓ Used to elicit and structure all information requirements
 - B. **Logical Design:** a higher-level conceptual abstraction with selected specific data model to implement the data structure.
 - ✓ It is particular DBMS independent and with no other physical considerations.
 - C. **Physical Design:** physical implementation of the logical design of the database with respect to internal storage and file structure of the database for the selected DBMS.
 - ✓ To develop all technology and organizational specification.
4. **Implementation:** the testing and deployment of the designed database for use.
5. **Operation and Support:** administering and maintaining the operation of the database system and providing support to users. Tuning the database operations for best performance.

3.2 The Entity Relationship (ER) Model

The ER model defines the conceptual view of a database. It works around real-world entities and the associations among them. At view level, the ER model is considered a good option for designing databases.

3.2.1 Entity

An entity can be a real-world object, either animate or inanimate, that can be easily identifiable. For example, in a school database, students, teachers, classes, and courses offered can be considered as entities. All these entities have some attributes or properties that give them their identity.

An entity set is a collection of similar types of entities. An entity set may contain entities with attribute sharing similar values. For example, a student's set may contain all the students of a school; likewise, a teachers set may contain all the teachers of a school from all faculties. Entity sets need not be disjoint.

3.2.2 Attributes

Entities are represented by means of their properties, called **attributes**. All attributes have values. For example, a student entity may have name, class, and age as attributes.

There exists a domain or range of values that can be assigned to attributes. For example, a student's name cannot be a numeric value. It has to be alphabetic. A student's age cannot be negative, etc.

3.2.2.1 Types of Attributes

- ✓ **Simple attribute** – Simple attributes are atomic values, which cannot be divided further. For example, a student's phone number is an atomic value of 10 digits.
- ✓ **Composite attribute** – Composite attributes are made of more than one simple attribute. For example, a student's complete name may have first_name and last_name.

- ✓ **Derived attribute** – Derived attributes are the attributes that do not exist in the physical database, but their values are derived from other attributes present in the database. For example, average_salary in a department should not be saved directly in the database, instead it can be derived. For another example, age can be derived from data_of_birth.
- ✓ **Single-value attribute** – Single-value attributes contain single value. For example – Social_Security_Number.
- ✓ **Multi-value attribute** – multi-value attributes may contain more than one values. For example, a person can have more than one phone number, email_address, etc.

These attribute types can come together in a way like –

- ✓ simple single-valued attributes
- ✓ simple multi-valued attributes
- ✓ composite single-valued attributes
- ✓ composite multi-valued attributes

3.2.3 Entity-Set and Keys

Key is an attribute or collection of attributes that uniquely identifies an entity among entity set.

For example, the roll_number of a student makes him/her identifiable among students.

- ✓ **Super Key** – A set of attributes (one or more) that collectively identifies an entity in an entity set.
- ✓ **Candidate Key** – A minimal super key is called a candidate key. An entity set may have more than one candidate key.
- ✓ **Primary Key** – A primary key is one of the candidate keys chosen by the database designer to uniquely identify the entity set.

3.2.4 Relationship

The association among entities is called a relationship. For example, an employee **works_at** a department, a student **enrolls** in a course. Here, Works_at and enrolls are called relationships.

3.2.5 Relationship Set

A set of relationships of similar type is called a relationship set. Like entities, a relationship too can have attributes. These attributes are called **descriptive attributes**.

3.2.6 Degree of Relationship

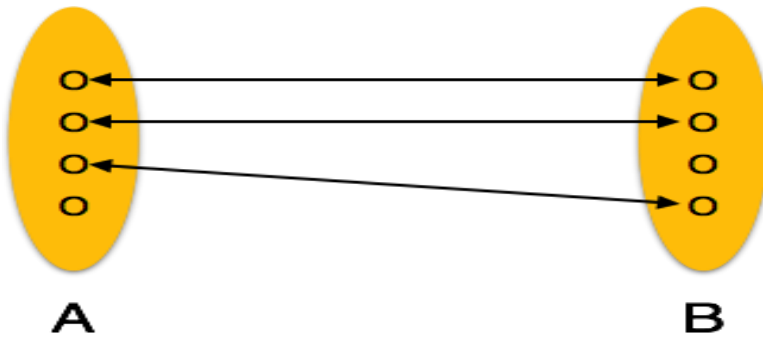
The number of participating entities in a relationship defines the degree of the relationship.

- ✓ Binary = degree 2
- ✓ Ternary = degree 3
- ✓ n-ary = degree

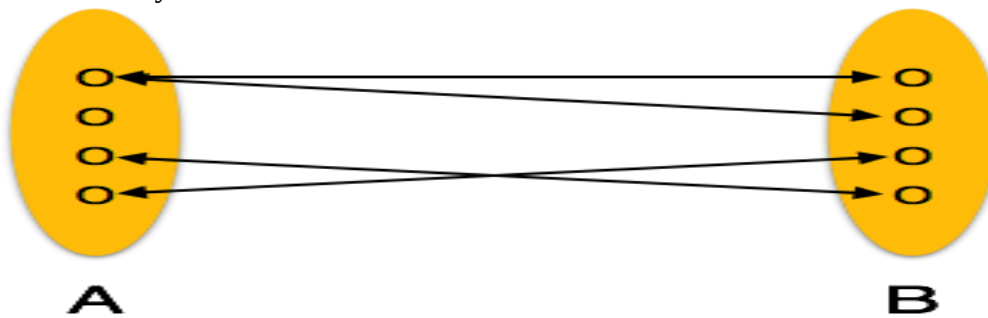
3.2.7 Mapping Cardinalities

Cardinality defines the number of entities in one entity set, which can be associated with the number of entities of other set via relationship set.

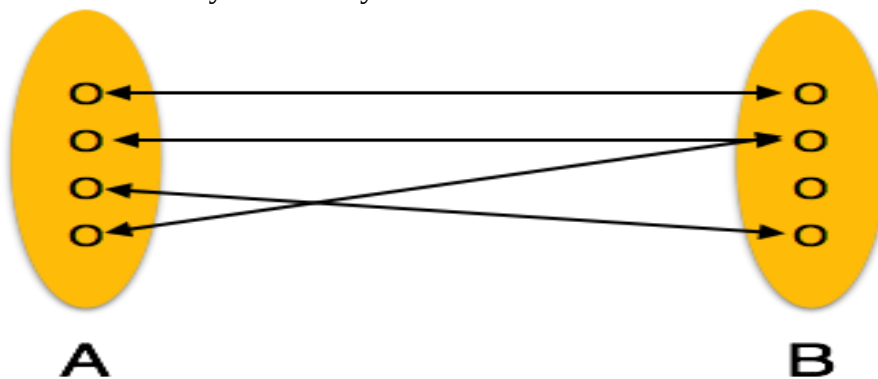
- ✓ **One-to-one** – One entity from entity set A can be associated with at most one entity of entity set B and vice versa.



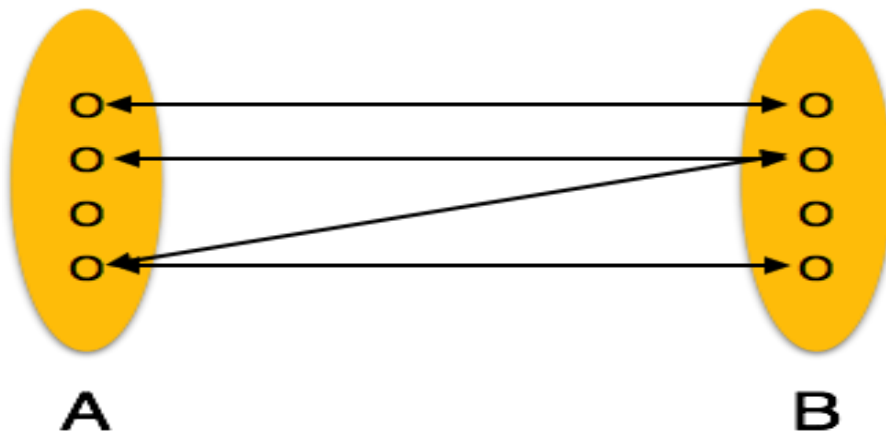
- ✓ **One-to-many** – One entity from entity set A can be associated with more than one entity of entity set B however an entity from entity set B, can be associated with at most one entity.



- ✓ **Many-to-one** – More than one entity from entity set A can be associated with at most one entity of entity set B, however an entity from entity set B can be associated with more than one entity from entity set A.



- ✓ **Many-to-many** – One entity from A can be associated with more than one entity from B and vice versa.



3.3 ER Diagram Representation

Let us now learn how the ER Model is represented by means of an ER diagram. Any object, for example, entities, attributes of an entity, relationship sets, and attributes of relationship sets, can be represented with the help of an ER diagram.

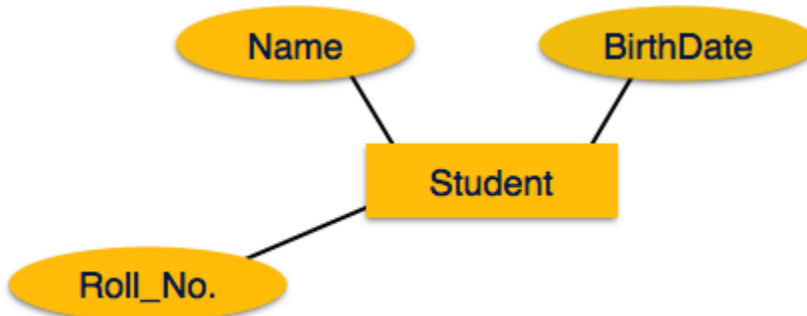
3.3.1 Entity

Entities are represented by means of rectangles. Rectangles are named with the entity set they represent.

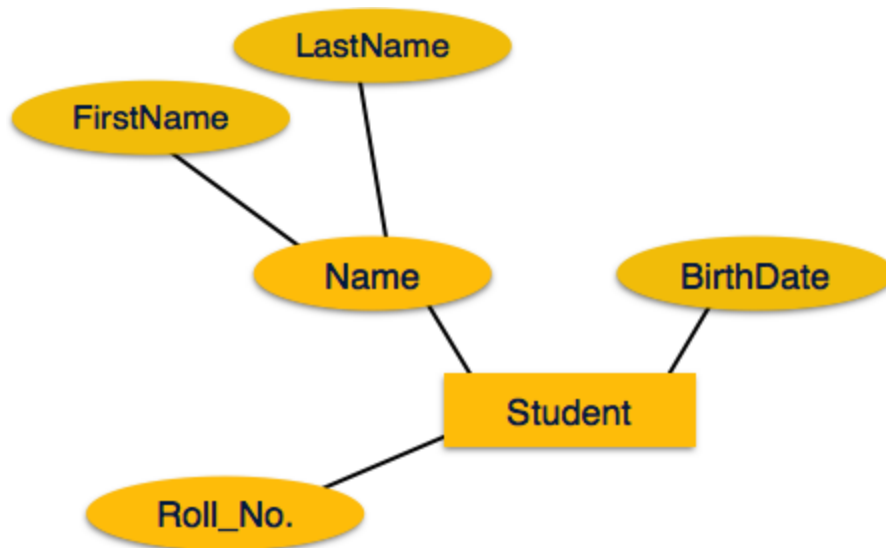


3.3.2 Attributes

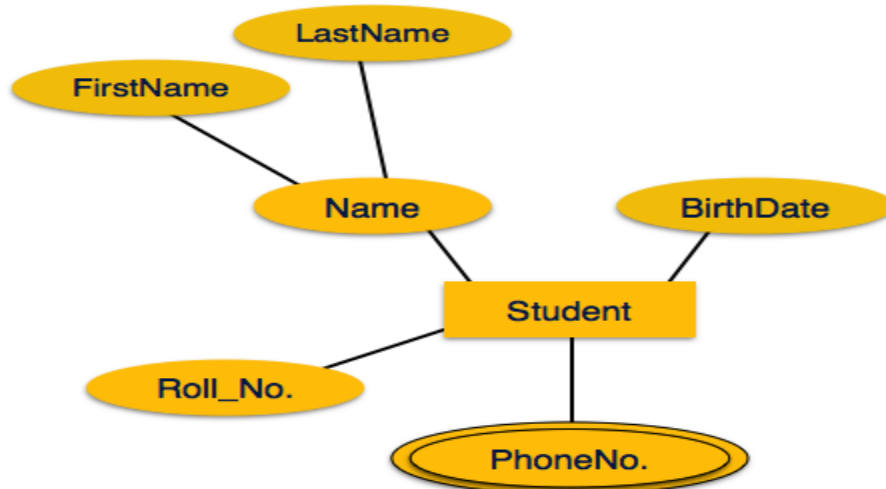
Attributes are the properties of entities. Attributes are represented by means of ellipses. Every ellipse represents one attribute and is directly connected to its entity (rectangle).



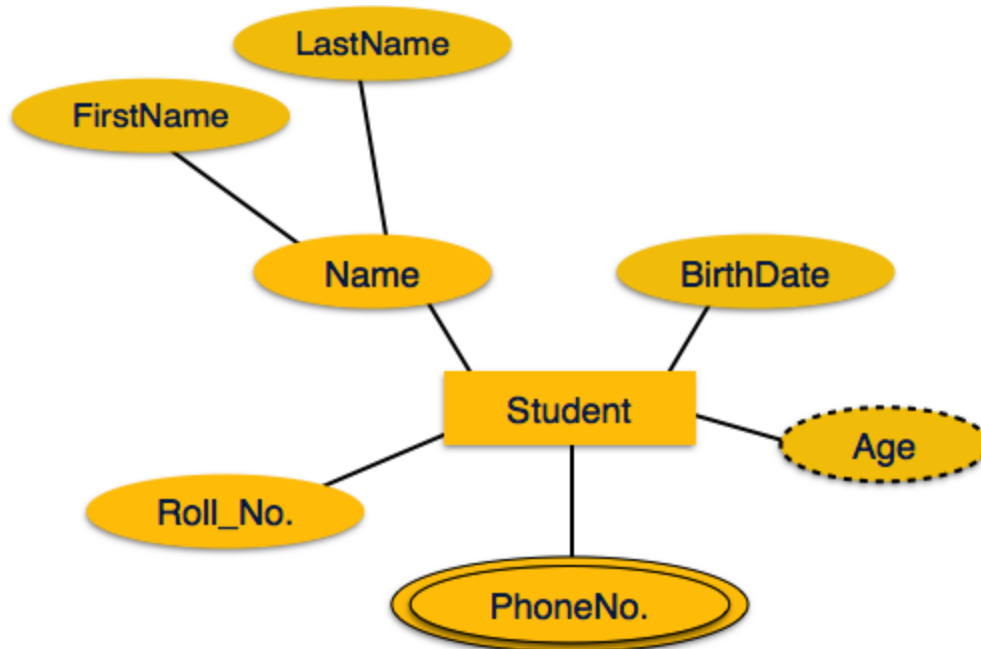
If the attributes are **composite**, they are further divided in a tree like structure. Every node is then connected to its attribute. That is, composite attributes are represented by ellipses that are connected with an ellipse.



Multivalued attributes are depicted by double ellipse.



Derived attributes are depicted by dashed ellipse.



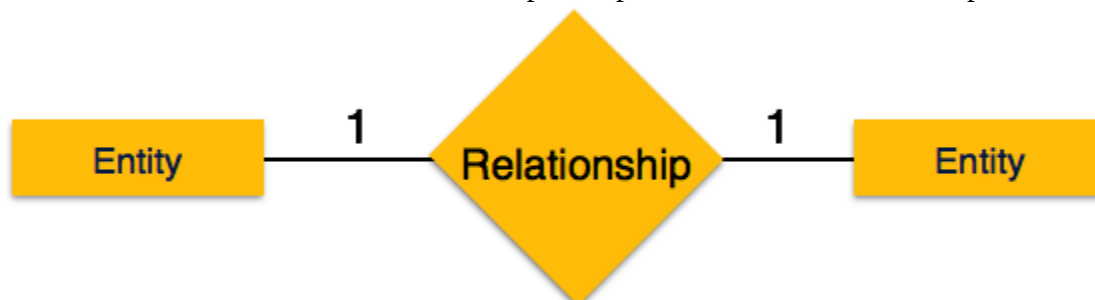
3.3.3 Relationship

Relationships are represented by diamond-shaped box. Name of the relationship is written inside the diamond-box. All the entities (rectangles) participating in a relationship, are connected to it by a line.

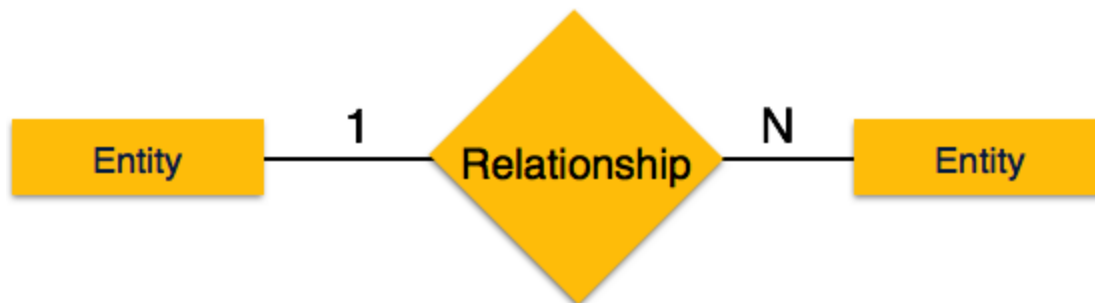
3.3.3.1 Binary Relationship and Cardinality

A relationship where two entities are participating is called a **binary relationship**. Cardinality is the number of instance of an entity from a relation that can be associated with the relation.

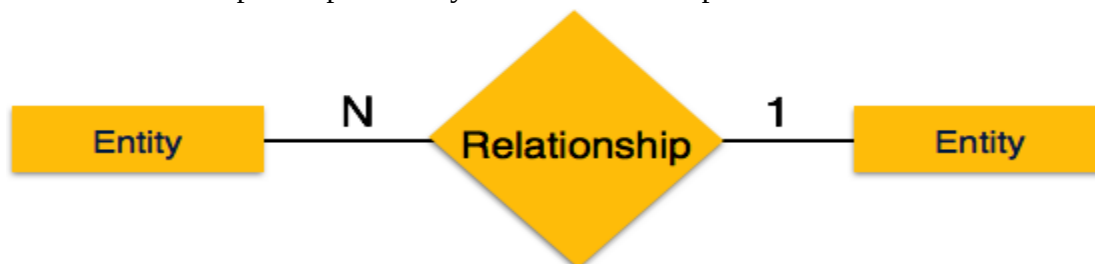
- ✓ **One-to-one** – When only one instance of an entity is associated with the relationship, it is marked as '1:1'. The following image reflects that only one instance of each entity should be associated with the relationship. It depicts one-to-one relationship.



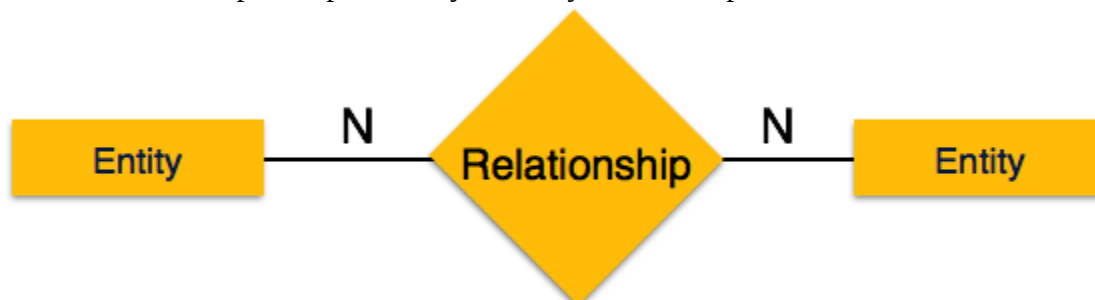
- ✓ **One-to-many** – When more than one instance of an entity is associated with a relationship, it is marked as '1:N'. The following image reflects that only one instance of entity on the left and more than one instance of an entity on the right can be associated with the relationship. It depicts one-to-many relationship.



- ✓ **Many-to-one** – When more than one instance of entity is associated with the relationship, it is marked as 'N:1'. The following image reflects that more than one instance of an entity on the left and only one instance of an entity on the right can be associated with the relationship. It depicts many-to-one relationship.

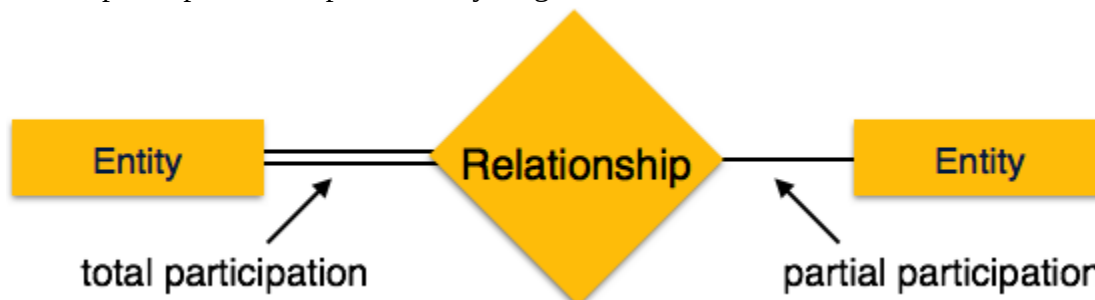


- ✓ **Many-to-many** – The following image reflects that more than one instance of an entity on the left and more than one instance of an entity on the right can be associated with the relationship. It depicts many-to-many relationship.



3.3.3.2 Participation Constraints

- ✓ **Total Participation** – Each entity is involved in the relationship. Total participation is represented by double lines.
- ✓ **Partial participation** – Not all entities are involved in the relationship. Partial participation is represented by single lines.



3.4 Mapping ER-models to relational tables

ER Model, when conceptualized into diagrams, gives a good overview of entity-relationship, which is easier to understand. ER diagrams can be mapped to relational schema, that is, it is possible to create relational schema using ER diagram. We cannot import all the ER constraints into relational model, but an approximate schema can be generated.

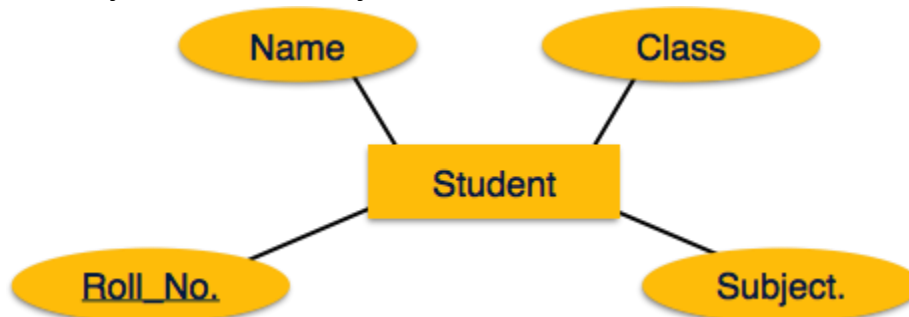
There are several processes and algorithms available to convert ER Diagrams into Relational Schema. Some of them are automated and some of them are manual. We may focus here on the mapping diagram contents to relational basics.

ER diagrams mainly comprise of –

- ✓ Entity and its attributes
- ✓ Relationship, which is association among entities.

3.4.1 Mapping Entity

An entity is a real-world object with some attributes.

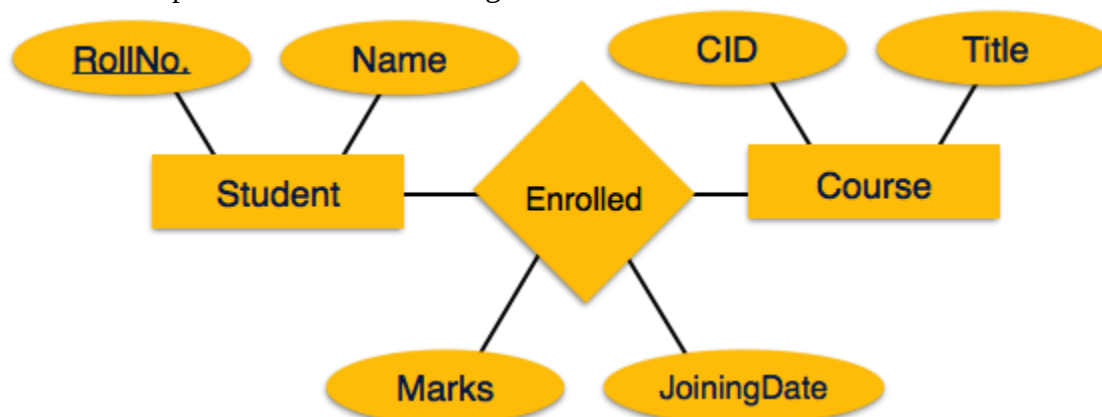


Mapping Process (Algorithm)

- ✓ Create table for each entity.
- ✓ Entity's attributes should become fields of tables with their respective data types.
- ✓ Declare primary key.

3.4.2 Mapping Relationship

A relationship is an association among entities.



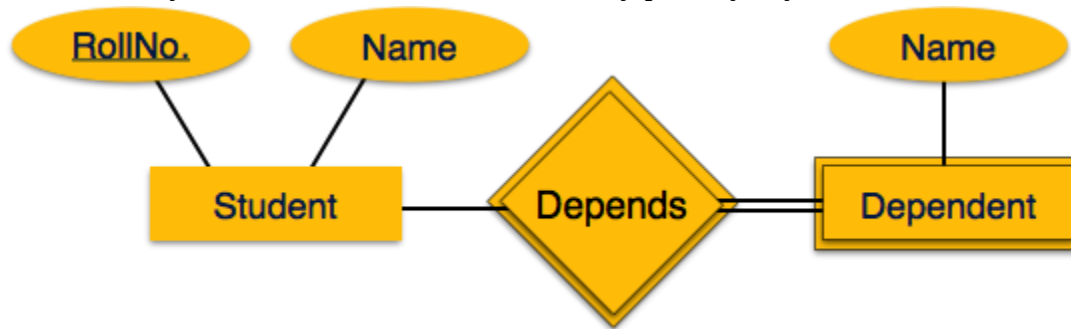
Mapping Process

- ✓ Create table for a relationship.

- ✓ Add the primary keys of all participating Entities as fields of table with their respective data types.
- ✓ If relationship has any attribute, add each attribute as field of table.
- ✓ Declare a primary key composing all the primary keys of participating entities.
- ✓ Declare all foreign key constraints.

3.4.3 Mapping Weak Entity Sets

A weak entity set is one which does not have any primary key associated with it.

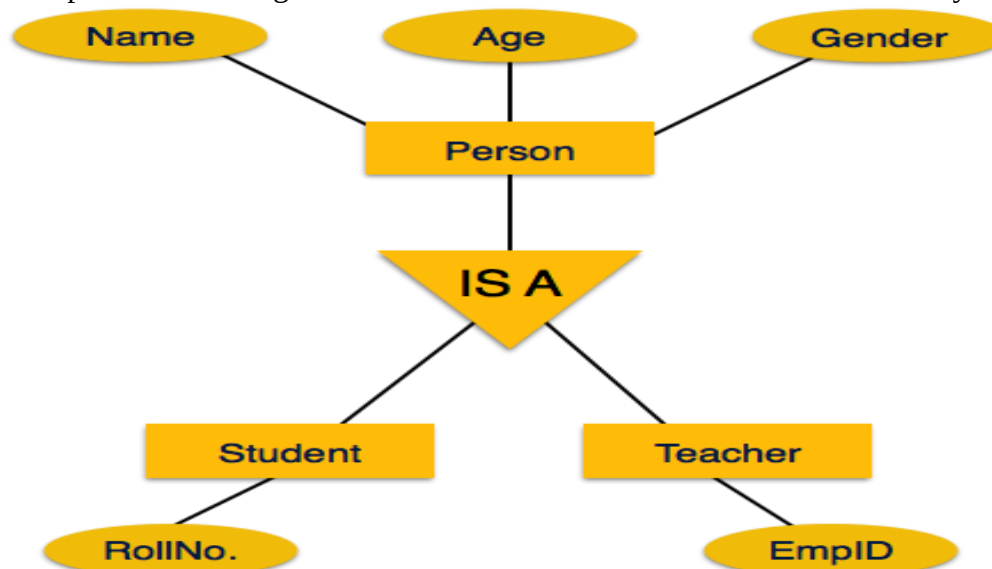


Mapping Process

- ✓ Create table for weak entity set.
- ✓ Add all its attributes to table as field.
- ✓ Add the primary key of identifying entity set.
- ✓ Declare all foreign key constraints.

3.4.4 Mapping Hierarchical Entities

ER specialization or generalization comes in the form of hierarchical entity sets.



Mapping Process

- ✓ Create tables for all higher-level entities.
- ✓ Create tables for lower-level entities.
- ✓ Add primary keys of higher-level entities in the table of lower-level entities.
- ✓ In lower-level tables, add all other attributes of lower-level entities.

- ✓ Declare primary key of higher-level table and the primary key for lower-level table.
- ✓ Declare foreign key constraints.

3.5 Enhanced Entity Relationship (EER) Model

EER Model

EER is a high-level data model that incorporates the extensions to the original ER model.

It is a diagrammatic technique for displaying the following concepts

- ✓ Sub Class and Super Class
- ✓ Specialization and Generalization
- ✓ Union or Category
- ✓ Aggregation

These concepts are used when they come in EER schema and the resulting schema diagrams called as EER Diagrams.

- ✓ Features of EER Model
- ✓ EER creates a design more accurate to database schemas.
- ✓ It reflects the data properties and constraints more precisely.
- ✓ It includes all modeling concepts of the ER model.
- ✓ Diagrammatic technique helps for displaying the EER schema.
- ✓ It includes the concept of specialization and generalization.
- ✓ It is used to represent a collection of objects that is union of objects of different of different entity types.

3.5.1 Sub Class and Super Class

- ✓ Sub class and Super class relationship leads the concept of Inheritance.
- ✓ The relationship between sub class and super class is denoted with  symbol.

1. Super Class

- ✓ Super class is an entity type that has a relationship with one or more subtypes.
 - ✓ An entity cannot exist in database merely by being member of any super class.
- For example:** Shape super class is having sub groups as Square, Circle, Triangle.

2. Sub Class

- ✓ Sub class is a group of entities with unique attributes.
 - ✓ Sub class inherits properties and attributes from its super class.
- For example:** Square, Circle, Triangle are the sub class of Shape super class.

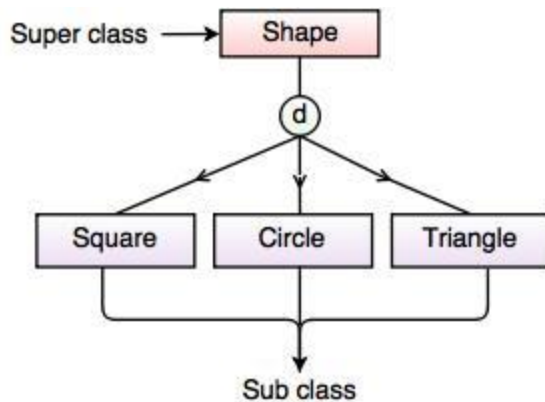


Fig. Super class/Sub class Relationship

3.5.2 Specialization and Generalization

1. Generalization

- ✓ Generalization is the process of generalizing the entities which contain the properties of all the generalized entities.
- ✓ It is a bottom approach, in which two lower-level entities combine to form a higher-level entity.
- ✓ Generalization is the reverse process of Specialization.
- ✓ It defines a general entity type from a set of specialized entity type.
- ✓ It minimizes the difference between the entities by identifying the common features.

For example:

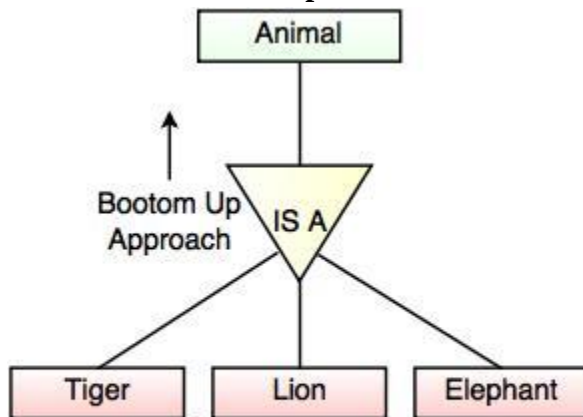


Fig. Generalization

In the above example, Tiger, Lion, Elephant can all be generalized as Animals.

2. Specialization

- ✓ Specialization is a process that defines a group entity which is divided into sub groups based on their characteristic.
- ✓ It is a top-down approach, in which one higher entity can be broken down into two lower-level entity.

- ✓ It maximizes the difference between the members of an entity by identifying the unique characteristic or attributes of each member.
- ✓ It defines one or more sub class for the super class and also forms the superclass/subclass relationship.

For example

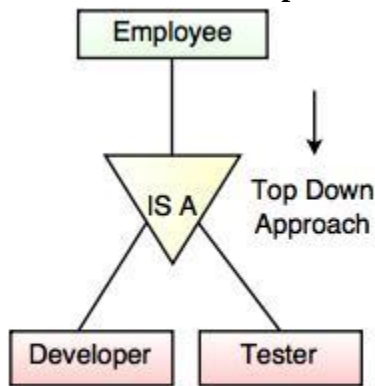


Fig. Specialization

In the above example, Employee can be specialized as Developer or Tester, based on what role they play in an organization.

3.5.3 Category or Union

- ✓ Category represents a single super class or sub class relationship with more than one super class.
- ✓ It can be a total or partial participation.

For example, Car booking, Car owner can be a person, a bank (holds a possession on a Car) or a company. Category (sub class) → Owner is a subset of the union of the three super classes → Company, Bank, and Person. A Category member must exist in at least one of its super classes.

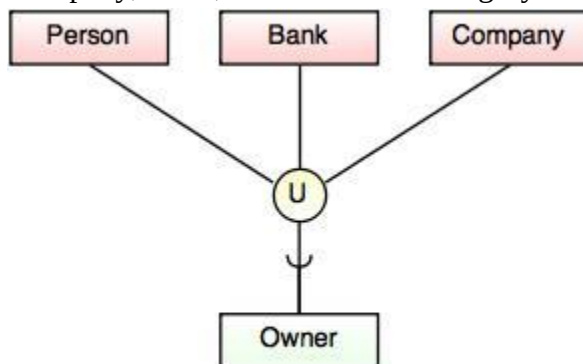


Fig. Categories (Union Type)

3.5.4 Aggregation

- ✓ Aggregation is a process that represent a relationship between a whole object and its component parts.
- ✓ It abstracts a relationship between objects and viewing the relationship as an object.

✓ It is a process when two entity is treated as a single entity.

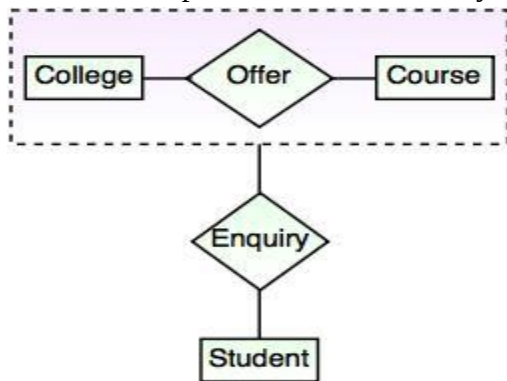


Fig. Aggregation

In the above example, the relation between College and Course is acting as an Entity in Relation with Student.

3.6 The Relational Database Model

Relational data model is the primary data model, which is used widely around the world for data storage and processing. This model is simple and it has all the properties and capabilities required to process data with storage efficiency.

3.6.1 Concepts

Tables – In relational data model, relations are saved in the format of Tables. This format stores the relation among entities. A table has rows and columns, where rows represent records and columns represent the attributes.

Tuple – A single row of a table, which contains a single record for that relation is called a tuple.

Relation instance – A finite set of tuples in the relational database system represents relation instance. Relation instances do not have duplicate tuples.

Relation schema – A relation schema describes the relation's name (table name), attributes, and their names.

Relation key – Each row has one or more attributes, known as relation key, which can identify the row in the relation (table) uniquely.

Attribute domain – Every attribute has some pre-defined value scope, known as attribute domain.

3.6.2 Constraints

Every relation has some conditions that must hold for it to be a valid relation. These conditions are called **Relational Integrity Constraints**. There are three main integrity constraints –

- ✓ Key constraints
- ✓ Domain constraints
- ✓ Referential integrity constraints

3.6.3 Key Constraints

There must be at least one minimal subset of attributes in the relation, which can identify a tuple uniquely. This minimal subset of attributes is called **key** for that relation. If there are more than one such minimal subsets, these are called ***candidate keys***.

Key constraints force that –

- ✓ in a relation with a key attribute, no two tuples can have identical values for key attributes.
- ✓ a key attribute cannot have NULL values.

Key constraints are also referred to as Entity Constraints.

3.6.4 Domain Constraints

Attributes have specific values in real-world scenario. For example, age can only be a positive integer. The same constraints have been tried to employ on the attributes of a relation. Every attribute is bound to have a specific range of values. For example, age cannot be less than zero and telephone numbers cannot contain a digit outside 0-9.

3.6.5 Referential integrity Constraints

Referential integrity constraints work on the concept of Foreign Keys. A foreign key is a key attribute of a relation that can be referred in other relation.

Referential integrity constraint states that if a relation refers to a key attribute of a different or same relation, then that key element must exist.

CHAPTER FOUR

4 Functional Dependency and Normalization

4.1 Functional Dependency

Functional dependencies are relationships between attributes in a database. They describe how one attribute is dependent on another attribute. For example, consider a database of employee records. The employee's ID number might be functionally dependent on their name because the name determines the ID number. In this case, we would say that the ID number is functionally dependent on the name.

Functional dependencies can be used to design a database in a way that eliminates redundancy and ensures data integrity. For example, consider a database that stores employee records and the departments they work in. If we store the department name for each employee, we might end up with several copies of the same department name.

This would be redundant and would take up unnecessary space in the database. Instead, we can use functional dependencies to store the department name only once and use the employee's ID number to determine which department they work in. This reduces redundancy and makes the database more efficient.

4.2 Normalization

Normalization is the process of organizing a database to reduce redundancy and dependency. It is important because it helps to eliminate data inconsistencies and ensures that the data is stored in a logical and organized way.

For example, consider a database that stores customer information and the products they have purchased. If we store the product names with each customer record, we might end up with several copies of the same product name. This would be redundant and would take up unnecessary space in the database. Instead, we can use normalization to create a separate table for products and store the product names only once. This reduces redundancy and makes the database more efficient.

There are several normal forms that can be used to normalize a database. The most common normal forms are the first, second, and third normal forms.

4.2.1 First normal form (1NF)

The first normal form (1NF) is a basic level of normalization. To be in 1NF, a table must meet the following criteria –

It must contain only atomic values. An atomic value is a single value that cannot be further broken down. For example, a name is an atomic value, but an address is not because it can be broken down into separate values for the street, city, state, and zip code.

It must not contain repeating groups. A repeating group is a set of values that are repeated within a single record. For example, if a table contains a field for phone numbers, it should not contain multiple phone numbers within the same field. Instead, there should be separate fields for each phone number.

4.2.2 Second normal form (2NF)

The second normal form (2NF) is a higher level of normalization. To be in 2NF, a table must meet the following criteria –

- ✓ It must be in 1NF.
- ✓ It must not have any partial dependencies. A partial dependency occurs when a non-key attribute is dependent on only a part of the primary key. For example, consider a table with the following attributes: EmployeeID (primary key), EmployeeName, and DepartmentID. If the DepartmentID is dependent on the EmployeeID, but not on the EmployeeName, there is a partial dependency. To eliminate this dependency, we would create a separate table for departments and store the DepartmentID and DepartmentName in that table.

4.2.3 Third normal form (3NF)

The third normal form (3NF) is a higher level of normalization. To be in 3NF, a table must meet the following criteria –

- ✓ It must be in 2NF.
- ✓ It must not have any transitive dependencies. A transitive dependency occurs when an attribute is dependent on another attribute that is not the primary key. For example, consider a table with the following attributes: EmployeeID (primary key), EmployeeName, and ManagerID. If the ManagerID is dependent on the EmployeeID, which is the primary key, there is no transitive dependency. However, if the ManagerID is dependent on the EmployeeName, which is not the primary key, there is a transitive dependency. To eliminate this dependency, we would create a separate table for managers and store the ManagerID and ManagerName in that table.

Real-life Examples

To better understand these concepts, let's look at some real-life examples of functional dependencies and normalization.

Example 1

Consider a database of customer orders for an online store. The following table stores information about each order –

OrderID	CustomerID	ProductID	Quantity
1	1	10	2
2	1	11	1

3	2	10	3
---	---	----	---

In this table, the OrderID is the primary key and the CustomerID and ProductID are foreign keys. The Quantity attribute is dependent on the OrderID, because it determines the quantity of each product in the order.

This table is in 1NF because it contains only atomic values and does not have any repeating groups. However, it is not in 2NF because the Quantity attribute is dependent on the OrderID, which is only a part of the primary key (OrderID, ProductID). To eliminate this partial dependency, we can create a separate table for order details and store the OrderID, ProductID, and Quantity in that table.

OrderID	ProductID	Quantity
1	10	2
1	11	1
3	10	3

Example 2

Consider a database of employee records for a company. The following table stores information about each employee

EmployeeID	EmployeeName	ManagerID	DepartmentID
1	John Smith	3	1
2	Jane Doe	3	1
3	Bob Johnson	4	2
4	Mary Williams	NULL	2

In this table, the EmployeeID is the primary key and the ManagerID and DepartmentID are foreign keys. The ManagerID is dependent on the EmployeeID, because it determines the employee's

manager. The DepartmentID is dependent on the ManagerID, because it determines the department the employee works in.

This table is in 2NF because it is in 1NF and does not have any partial dependencies. However, it is not in 3NF because the DepartmentID is dependent on the ManagerID, which is not the primary key. To eliminate this transitive dependency, we can create a separate table for departments and store the DepartmentID and DepartmentName in that table. We can then update the employees table to store the DepartmentID as a foreign key.

EmployeeID	EmployeeName	ManagerID	DepartmentID
1	John Smith	3	1
2	Jane Doe	3	1
3	Bob Johnson	4	2
4	Mary Williams	NULL	2
DepartmentID		DepartmentName	
1		Sales	
2		Marketing	

4.3 Conclusion

Functional dependencies and normalization are important concepts in relational database design. They help to eliminate redundancy and ensure data integrity by organizing the database in a logical and efficient way. By understanding these concepts and applying them to your database design, you can create a database that is efficient, effective, and easy to maintain.

CHAPTER FIVE

5 Record Storage and Primary File Organization

5.1 File Storage System

Databases are stored in file formats, which contain records. At physical level, the actual data is stored in electromagnetic format on some device. These storage devices can be broadly categorized into three types –

- ✓ **Primary Storage** – The memory storage that is directly accessible to the CPU comes under this category. CPU's internal memory (registers), fast memory (cache), and main memory (RAM) are directly accessible to the CPU, as they are all placed on the motherboard or CPU chipset. This storage is typically very small, ultra-fast, and volatile. Primary storage requires continuous power supply in order to maintain its state. In case of a power failure, all its data is lost.
- ✓ **Secondary Storage** – Secondary storage devices are used to store data for future use or as backup. Secondary storage includes memory devices that are not a part of the CPU chipset or motherboard, for example, magnetic disks, optical disks (DVD, CD, etc.), hard disks, flash drives, and magnetic tapes.
- ✓ **Tertiary Storage** – Tertiary storage is used to store huge volumes of data. Since such storage devices are external to the computer system, they are the slowest in speed. These storage devices are mostly used to take the back up of an entire system. Optical disks and magnetic tapes are widely used as tertiary storage.

5.1.1 Memory Hierarchy

A computer system has a well-defined hierarchy of memory. A CPU has direct access to its main memory as well as its inbuilt registers. The access time of the main memory is obviously less than the CPU speed. To minimize this speed mismatch, cache memory is introduced. Cache memory provides the fastest access time and it contains data that is most frequently accessed by the CPU.

The memory with the fastest access is the costliest one. Larger storage devices offer slow speed and they are less expensive, however they can store huge volumes of data as compared to CPU registers or cache memory.

5.1.2 Magnetic Disks

Hard disk drives are the most common secondary storage devices in present computer systems. These are called magnetic disks because they use the concept of magnetization to store information. Hard disks consist of metal disks coated with magnetizable material. These disks are placed vertically on a spindle. A read/write head moves in between the disks and is used to magnetize or de-magnetize the spot under it. A magnetized spot can be recognized as 0 (zero) or 1 (one).

Hard disks are formatted in a well-defined order to store data efficiently. A hard disk plate has many concentric circles on it, called **tracks**. Every track is further divided into **sectors**. A sector on a hard disk typically stores 512 bytes of data.

5.1.3 Redundant Array of Independent Disks

RAID or **Redundant Array of Independent Disks**, is a technology to connect multiple secondary storage devices and use them as a single storage media.

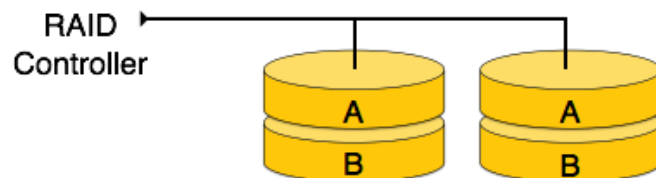
RAID consists of an array of disks in which multiple disks are connected together to achieve different goals. RAID levels define the use of disk arrays.

RAID 0



In this level, a striped array of disks is implemented. The data is broken down into blocks and the blocks are distributed among disks. Each disk receives a block of data to write/read in parallel. It enhances the speed and performance of the storage device. There is no parity and backup in Level 0.

RAID 1



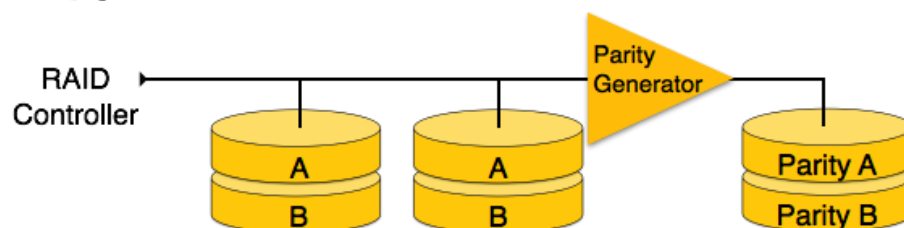
RAID 1 uses mirroring techniques. When data is sent to a RAID controller, it sends a copy of data to all the disks in the array. RAID level 1 is also called **mirroring** and provides 100% redundancy in case of a failure.

RAID 2



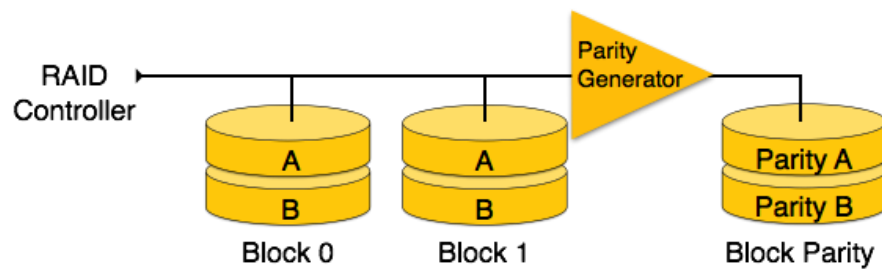
RAID 2 records Error Correction Code using Hamming distance for its data, striped on different disks. Like level 0, each data bit in a word is recorded on a separate disk and ECC codes of the data words are stored on a different set disk. Due to its complex structure and high cost, RAID 2 is not commercially available.

RAID 3



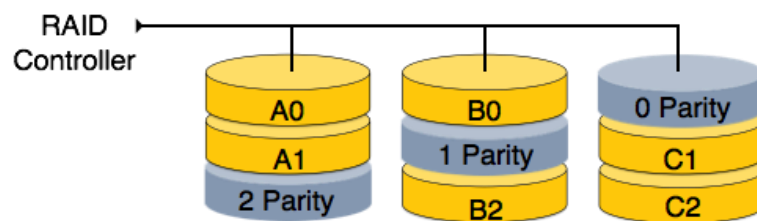
RAID 3 stripes the data onto multiple disks. The parity bit generated for data word is stored on a different disk. This technique makes it to overcome single disk failures.

RAID 4



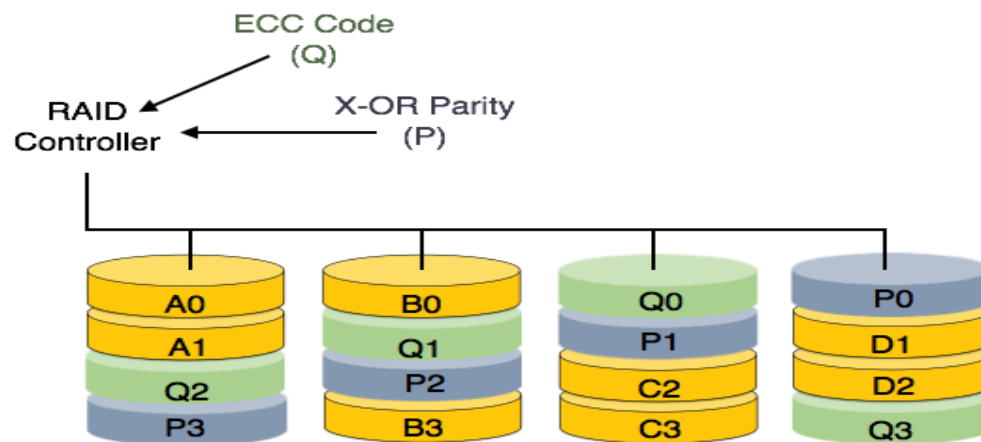
In this level, an entire block of data is written onto data disks and then the parity is generated and stored on a different disk. Note that level 3 uses byte-level striping, whereas level 4 uses block-level striping. Both level 3 and level 4 require at least three disks to implement RAID.

RAID 5



RAID 5 writes whole data blocks onto different disks, but the parity bits generated for data block stripe are distributed among all the data disks rather than storing them on a different dedicated disk.

RAID 6



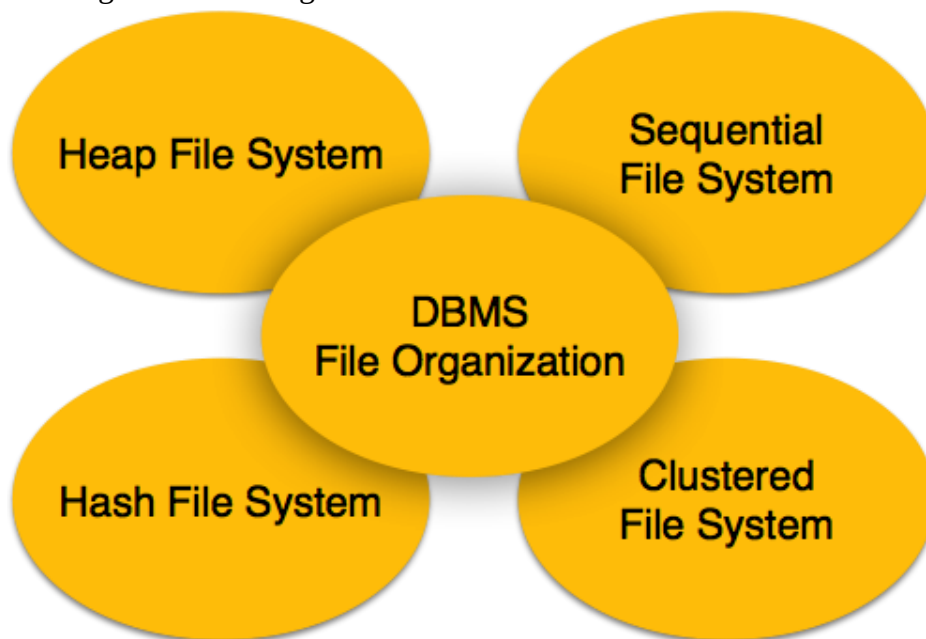
RAID 6 is an extension of level 5. In this level, two independent parities are generated and stored in distributed fashion among multiple disks. Two parities provide additional fault tolerance. This level requires at least four disk drives to implement RAID.

5.2 File Structure

Relative data and information is stored collectively in file formats. A file is a sequence of records stored in binary format. A disk drive is formatted into several blocks that can store records. File records are mapped onto those disk blocks.

5.2.1 File Organization

File Organization defines how file records are mapped onto disk blocks. We have four types of File Organization to organize file records –



5.2.1.1 Heap File Organization

When a file is created using Heap File Organization, the Operating System allocates memory area to that file without any further accounting details. File records can be placed anywhere in that memory area. It is the responsibility of the software to manage the records. Heap File does not support any ordering, sequencing, or indexing on its own.

5.2.1.2 Sequential File Organization

Every file record contains a data field (attribute) to uniquely identify that record. In sequential file organization, records are placed in the file in some sequential order based on the unique key field or search key. Practically, it is not possible to store all the records sequentially in physical form.

5.2.1.3 Hash File Organization

Hash File Organization uses Hash function computation on some fields of the records. The output of the hash function determines the location of disk block where the records are to be placed.

5.2.1.4 Clustered File Organization

Clustered file organization is not considered good for large databases. In this mechanism, related records from one or more relations are kept in the same disk block, that is, the ordering of records is not based on primary key or search key.

5.2.2 File Operations

Operations on database files can be broadly classified into two categories –

- ✓ Update Operations
- ✓ Retrieval Operations

Update operations change the data values by insertion, deletion, or update. Retrieval operations, on the other hand, do not alter the data but retrieve them after optional conditional filtering. In both types of operations, selection plays a significant role. Other than creation and deletion of a file, there could be several operations, which can be done on files.

- **Open** – A file can be opened in one of the two modes, **read mode** or **write mode**. In read mode, the operating system does not allow anyone to alter data. In other words, data is read only. Files opened in read mode can be shared among several entities. Write mode allows data modification. Files opened in write mode can be read but cannot be shared.
- **Locate** – Every file has a file pointer, which tells the current position where the data is to be read or written. This pointer can be adjusted accordingly. Using find (seek) operation, it can be moved forward or backward.
- **Read** – By default, when files are opened in read mode, the file pointer points to the beginning of the file. There are options where the user can tell the operating system where to locate the file pointer at the time of opening a file. The very next data to the file pointer is read.
- **Write** – User can select to open a file in write mode, which enables them to edit its contents. It can be deletion, insertion, or modification. The file pointer can be located at the time of opening or can be dynamically changed if the operating system allows to do so.
- **Close** – This is the most important operation from the operating system's point of view. When a request to close a file is generated, the operating system
 - ✓ removes all the locks (if in shared mode),
 - ✓ saves the data (if altered) to the secondary storage media, and
 - ✓ releases all the buffers and file handlers associated with the file.

The organization of data inside a file plays a major role here. The process to locate the file pointer to a desired record inside a file varies based on whether the records are arranged sequentially or clustered.

5.2.3 Indexing

We know that data is stored in the form of records. Every record has a key field, which helps it to be recognized uniquely.

Indexing is a data structure technique to efficiently retrieve records from the database files based on some attributes on which the indexing has been done. Indexing in database systems is similar to what we see in books.

Indexing is defined based on its indexing attributes. Indexing can be of the following types –

- ✓ **Primary Index** – Primary index is defined on an ordered data file. The data file is ordered on a **key field**. The key field is generally the primary key of the relation.
- ✓ **Secondary Index** – Secondary index may be generated from a field which is a candidate key and has a unique value in every record, or a non-key with duplicate values.
- ✓ **Clustering Index** – Clustering index is defined on an ordered data file. The data file is ordered on a non-key field.

Ordered Indexing is of two types –

- ✓ Dense Index
- ✓ Sparse Index

5.2.3.1 Dense Index

In dense index, there is an index record for every search key value in the database. This makes searching faster but requires more space to store index records itself. Index records contain search key value and a pointer to the actual record on the disk.



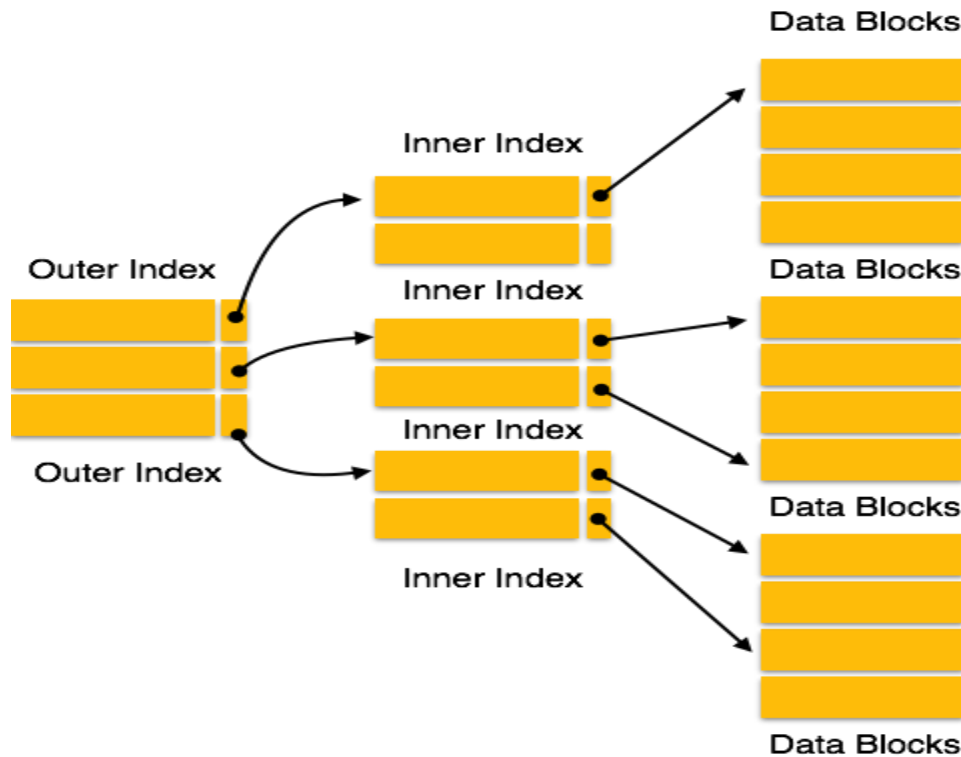
5.2.3.2 Sparse Index

In sparse index, index records are not created for every search key. An index record here contains a search key and an actual pointer to the data on the disk. To search a record, we first proceed by index record and reach at the actual location of the data. If the data we are looking for is not where we directly reach by following the index, then the system starts sequential search until the desired data is found.



5.2.3.3 Multilevel Index

Index records comprise search-key values and data pointers. Multilevel index is stored on the disk along with the actual database files. As the size of the database grows, so does the size of the indices. There is an immense need to keep the index records in the main memory so as to speed up the search operations. If single-level index is used, then a large size index cannot be kept in memory which leads to multiple disk accesses.



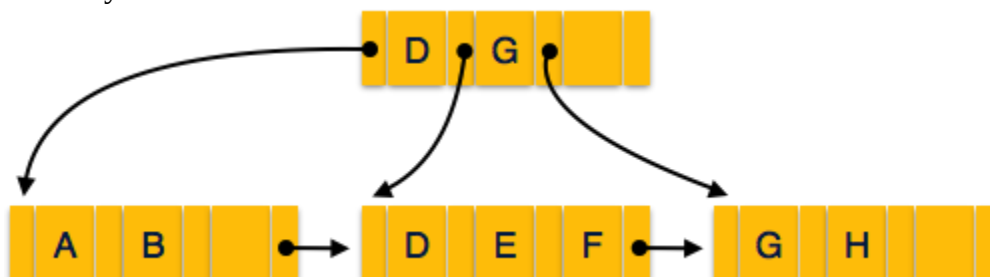
Multi-level Index helps in breaking down the index into several smaller indices in order to make the outermost level so small that it can be saved in a single disk block, which can easily be accommodated anywhere in the main memory.

5.2.3.4 B⁺ Tree

A B⁺ tree is a balanced binary search tree that follows a multi-level index format. The leaf nodes of a B⁺ tree denote actual data pointers. B⁺ tree ensures that all leaf nodes remain at the same height, thus balanced. Additionally, the leaf nodes are linked using a link list; therefore, a B⁺ tree can support random access as well as sequential access.

Structure of B⁺ Tree

Every leaf node is at equal distance from the root node. A B⁺ tree is of the order **n** where **n** is fixed for every B⁺ tree.



Internal nodes

- ✓ Internal (non-leaf) nodes contain at least $\lceil n/2 \rceil$ pointers, except the root node.
- ✓ At most, an internal node can contain **n** pointers.

Leaf nodes

- ✓ Leaf nodes contain at least $\lceil n/2 \rceil$ record pointers and $\lceil n/2 \rceil$ key values.
- ✓ At most, a leaf node can contain n record pointers and n key values.
- ✓ Every leaf node contains one block pointer **P** to point to next leaf node and forms a linked list.

B⁺ Tree Insertion

- ✓ B⁺ trees are filled from bottom and each entry is done at the leaf node.
- ✓ **If a leaf node overflows**
 - Split node into two parts.
 - Partition at $i = \lfloor (m+1)/2 \rfloor$.
 - First i entries are stored in one node.
 - Rest of the entries ($i+1$ onwards) are moved to a new node.
 - i^{th} key is duplicated at the parent of the leaf.
- ✓ **If a non-leaf node overflow**
 - Split node into two parts.
 - Partition the node at $i = \lfloor (m+1)/2 \rfloor$.
 - Entries up to i are kept in one node.
 - Rest of the entries are moved to a new node.

B⁺ Tree Deletion

- ✓ B⁺ tree entries are deleted at the leaf nodes.
- ✓ The target entry is searched and deleted.
 - If it is an internal node, delete and replace with the entry from the left position.
- ✓ After deletion, underflow is tested,
 - If underflow occurs, distribute the entries from the nodes left to it.
- ✓ If distribution is not possible from left, then
 - Distribute from the nodes right to it.
- ✓ If distribution is not possible from left or from right, then
 - Merge the node with left and right to it.

CHAPTER SIX

6 The Relational Algebra and Relational Calculus

6.1 Relational Algebra

Relational Algebra is a procedural query language that consists of a set of operations that take one or two relations as input and produce a new relation as a result. The algebra operations enable a user to retrieve specific request on a relational model. The operations that produce a new relation can be further manipulated using operations of the relation algebra. The sequence of the relational algebra that produces new relation forms a **relational algebra expression**.

6.1.1 Fundamental Operations of Relational Algebra

The core relational algebra that has traditionally been thought of as the relational algebra consists of the Fundamental operations that can be grouped into two based on the number of relation operands of the operator. These are:

Unary Operators.

- ✓ Selection (σ)
- ✓ Projection (Π)
- ✓ Rename(ρ)

Binary Operators

- ✓ Product (Cartesian Product) ($\times$)
- ✓ Union ($\cup$)
- ✓ Difference ($-$)

The binary operators listed above are also known as set operators as they are derived from the set theory.

6.1.1.1 Unary Operations

Select Operation

The select operation selects a subset of tuples from a relation instance that satisfies a given predicate (condition).

It is denoted by: $\sigma_C(R)$

Where σ represents the SELECT operator, C is a boolean expression of the select condition, and R is the relation or relational algebra expression.

For example

$\sigma_{\text{subject} = \text{"database"}}(\text{Books})$

Output – Selects tuples from books where subject is 'database'.

$\sigma_{\text{subject} = \text{"database"} \text{ and } \text{price} = \text{"450"}}(\text{Books})$

Output – Selects tuples from books where subject is 'database' and 'price' is 450.

$\sigma_{\text{subject} = \text{"database"} \text{ and } \text{price} = \text{"450"} \text{ or } \text{year} > \text{"2010"}}(\text{Books})$

Output – Selects tuples from books where subject is 'database' and 'price' is 450 or those books published after 2010.

Project Operation

While the select operation is picking certain rows from a relation, projection operation forms a new relation by picking certain columns in the relation.

It is denoted by: $\Pi(\mathbf{R})$, Where Π represents the PROJECT operator and $\mathbf{A}$ is a set of attributes in the relation $\mathbf{R}$.

For example

$\Pi_{\text{subject, author}}(\text{Books})$

Selects and projects columns named as subject and author from the relation Books.

Rename Operation

Unlike relations in the relational model the new relations driven from the relational algebra expression do not have name that will allow us to refer to them in other expressions. The renaming operator can be used to explicitly rename resulting relations of an expression.

It is denoted by: $\rho_{S(A1,A2,...An)}(\mathbf{R})$

Where ρ represents the RENAME operator and S is a name for the new relation and $A, A, \dots A$ are new names for the attributes in the relation $\mathbf{R}$.

After the renaming the name of the relation and the attributes can be used as ordinary relation and attributes in a sequence of relational algebra expressions:

6.1.1.2 Binary Operations

Cartesian Product Operation

The Cartesian product operation (also known as Cross Product or Cross Join or Product) is binary set operation that generates a new relation from two relation in a combinatorial fashion. It is denoted by $\mathbf{R} \times \mathbf{S}$, Where $\times$ represents the PRODUCT operator and $\mathbf{R}$ and $\mathbf{S}$ are relations to be joined.

The product operation is the just like the product operation in set theory that maps each tuple in relation $\mathbf{R}$ with every tuple in $\mathbf{S}$.

Example

Consider the following relations $\mathbf{R}$ and $\mathbf{S}$, then $\mathbf{R} \times \mathbf{S}$ is given as shown to the right.

$\mathbf{R}$		$\mathbf{S}$			$\mathbf{R} \times \mathbf{S}$				
A	B	B	C	D	A	R.B	S.B	C	D
1	2	2	a	x	1	2	2	a	x
1	2	4	b	y	1	2	4	b	y
1	2	5	c	z	1	2	5	c	z
3	4	2	a	x	3	4	2	a	x
3	4	4	b	y	3	4	4	b	y
3	4	5	c	z	3	4	5	c	z

Union Operation

The union operation on R and S denoted by $R \cup S$ results a relation that includes all tuples either in R or in S or in both. Duplicates are eliminated from the result.

Intersection Operation

The intersection operation on R and S denoted by $R \cap S$ results a relation that includes all tuples in both R and S.

Set Difference Operation

The result of the set difference operation on R and S denoted by $R - S$ is the set of elements in R but not in S.

For the set operations (Union, Intersection, Set difference) the two relational operands R and S must have same type of tuples, this condition is known as **Union Compatibility**.

Two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_m)$ are said to be union compatible if

6.1.2 Additional Operations

The set of relational algebra operations $\{\sigma, \Pi, \rho, \bowtie, \cup, -\}$ is a complete set that the other original relational algebra operations such as intersection, join, division and assignment can be expressed as the sequence of the fundamental operations. In situation where the use of the fundamental operators result complex and lengthy expressions such operators are helpful to minimize the complexity of queries.

6.1.2.1 Natural Join Operation

In the Cartesian product $R \times S$ operation in the above example notice that a select operation is used to retrieve the desired tuples from the joined relation that generates $m \times n$ tuples where m and n are number of distinct tuples of R and S.

A frequent type of join connects two relations by:

- ✓ Equating attributes of the same name, and
- ✓ Projecting out one copy of each pair of equated attributes.

Such a join is known as **Natural Join** and it is denoted by: $R \bowtie S$, Where $\bowtie$ represents the NATURAL JOIN operator and R and S are relations to be joined.

The product operation is the just like the product operation in set theory that maps each tuple in relation are with every tuple in S.

Theta Join Operation

While the natural join enforces a join condition by equating similar attributes in the relations to be joined; a theta join joins relations to an arbitrary condition C . The notation for theta join is: $R \bowtie_C S$

The result of the theta operation is constructed by:

- ✓ Taking the product of R and S , and
- ✓ Selecting only those tuples satisfying the condition C .

As with the product operation the schema for the resulting operation of the theta join is the union of the schemas of R and S . (That is, the operation does not eliminate repeated columns in the two relations R and S if any).

Assignment Operation

The assignment operation denoted by $\leftarrow$ is similar to assignment operation in programming that helps to assign the result of a relational algebra from the right into a relation variable to the left. Subsequent assignment operations can be used to develop complex sequential queries, the intermediate assignment operations do not result any relation to the user.

Division Operation

The division operation denoted by $\div$ is suited to queries that include “universal quantification” or the phrase “for all”. A division operation is applied to two relations $R(Z) \div S(X)$, where $X \subseteq Z$ and the result is $T(Y)$ where $Y = Z - X$. For tuples t to appear in the result T , the values in t must appear in R in combination with every tuple in S .

The division operation can be expressed using the sequence of the fundamental operators as:

$T1 \leftarrow \Pi_Y(R)$

$T2 \leftarrow \Pi_Y((S \times T1) - R)$

$T \leftarrow T1 - T2$

6.1.3 Extended Operations

The basic relational algebra operations have been extended in several ways to enhance the expressive power of the original relational algebra. Some of the extended operations are:

- ✓ Outer Join
- ✓ Extended Projection
- ✓ Duplicate Elimination
- ✓ Aggregation and Grouping, ...

6.1.3.1 Outer Join Operation

The natural join operation $R \bowtie S$ results a tuple when there is a match to the common attributes of the tuples in the relations R and S. Such joins are known as **Inner Join** operations. However, there are cases when we want to have all the tuples from the participating relations and form the join when there is much. In such cases **outer join** operations can be used to keep all the tuples in R, or all those in S, or all those in both relations irrespective of they having matching tuples in their common attributes.

The three types of outer join operators are:

✓ Left Outer Join

Left outer join denoted by: $R \rightarrow \bowtie S$

Keeps every tuples in the left relation R and when there is no matching for tuples in R from tuples in S, the attributes of S are filled (padded) with NULL values.

✓ Right Outer Join

Right outer join denoted by: $R \bowtie \rightarrow S$

Similar to the left outer join operation it keeps all tuples in the right relation S and when there is no matching for tuples in S from tuples in R, the attributes of R are padded with NULL values.

✓ Full Outer Join

Full outer join denoted by: $R \rightarrow \bowtie \rightarrow S$

Keeps all tuples in both the left and right relations when no matching tuples are found, padding them with NULL values as needed.

6.2 Introduction to Relational Calculus

The relational algebra that is discussed so far is a procedural query language on a relational database model. A **relational calculus** however is a declarative and nonprocedural expression that specifies a retrieval request, and hence there is no description of how to evaluate the query in a relational calculus. Rather, a relational calculus expression specifies what to be retrieved.

- ✓ A relational calculus is classified into two as:
- ✓ Tuple Relational Calculus, and

6.2.1 Domain Relational Calculus

A query in a tuple relational calculus (tuple calculus) is expressed as: $\{ t \mid p(t) \}$.

Where, t is a tuple variable and p(t) a predicate (condition) that is to be true for the tuple t. Formulas in the predicate of the tuple calculus are composed of atoms, variables and quantifiers $\exists$ (existential quantifier) and $\forall$ (universal quantifier).

Domain Relational Calculus

A query in a domain relational calculus (domain calculus) uses domain variables that take on values from an attributes domain rather than values for an entire tuple. It is expressed as: $\{ \langle x_1, x_2, \dots, x_n \rangle \mid p(x_1, x_2, \dots, x_n) \}$ Where, $x_1, x_2, \dots, x_n$ represent domain variables and P is the predicate as in the case of tuple calculus. Formulas in the predicate are build in the same ways as the tuple calculus predicates.

CHAPTER SEVEN

7 Structured Query Language (SQL)

7.1 Introduction

Structured Query Language (SQL) is a query language that is standardized by the American National Standards Institute (ANSI) for most commercial relational database management systems (RDBMS). To retrieve or update information users execute '*queries*' (SQL Statements) to pull or modify the requested information from the database using criteria that is defined by the user.

Unfortunately, there are many different versions of the SQL language, but to be in compliance with the ANSI standard, they must support the same major keywords in a similar manner (such as SELECT, UPDATE, DELETE, INSERT, WHERE, and others). Most of the SQL database programs also have their own proprietary extensions in addition to the SQL standard such as TSQL of Microsoft SQL Server and PLSQL of Oracle! SQL supports data definition, query and update in Data Definition Language and Data Manipulation Language (DML)

7.1.1 SQL Data Definition Language (DDL)

The Data Definition Language (DDL) part of SQL permits database tables to be created or deleted. It can also define indexes (keys), specify links between tables, and impose constraints between database tables.

The most important DDL statements in SQL are:

- ✓ CREATE TABLE - creates a new database table.
- ✓ ALTER TABLE - alters (changes) a database table.
- ✓ DROP TABLE - deletes a database table.
- ✓ CREATE INDEX - creates an index (search key).
- ✓ DROP INDEX - deletes an index.

The DDL statements are used for a schema definition of a relational database.

7.1.2 SQL Data Manipulation Language (DML)

The Data Manipulation Language (DML) is part of the SQL syntax for executing queries to insert, retrieve, update, and delete records. The statements are;

- ✓ INSERT INTO - inserts new data into a database table.
- ✓ SELECT - extracts data from a database table.
- ✓ UPDATE - updates data in a database table.
- ✓ DELETE - deletes data from a database table.

The four most common commands are also known as SQL CRUD statements after the words Create, Read, Update and Delete data.

7.2 Schema Definition in SQL

SQL uses the following terms for the corresponding terms in relational model

- ✓ Table – Relation
- ✓ Column – Attribute
- ✓ Row – Tuple

7.2.1 Schema Creation and Modification

The CREATE SCHEMA command in the SQL statement is used to group database objects such as tables, views and permissions. The syntax for the command is:

CREATE SCHEMA <schema_name> AUTHORIZATION <owner>

<schema_name> is the name of the schema and <owner> identifies the user who is the owner of the schema.

Example

- ✓ CREATE SCHEMA swprjct AUTHORIZATION dbo

SQL statements that can be included as part of the CREATE SCHEMA statement are:

- ✓ CREATE TABLE statement
- ✓ CREATE VIEW statement
- ✓ GRANT statement
- ✓ CREATE INDEX statement (not supported in Microsoft SQL Server 2000)

While CREATE SCHEMA command groups database objects the CREATE DATABASE command in the SQL statement is used to create a new database and the corresponding files for storing the database. The syntax for the command is: CREATE DATABASE <database_name>

Example:

- ✓ CREATE DATABASE SWPRJCT

<database_name> is the name of the new database.

The command also has different optional parameters in different RDBMS that helps in specifying owner, file, growth, ...

7.2.2 Table Creation and Modification

The CREATE TABLE command in the SQL statement is used to specify a new relation in a database by giving it a name and listing its attributes.

The syntax for the command is:

```
CREATE TABLE <table_name> (  
<column_name> <data_type> {column_constraint},;  
<column_name> <data_type> {column_constraint}  
)
```

- <column_name> is the name of the column.

- <data_type> is the SQL supported data types: CHAR(n), VARCHAR(n), INT, SMALLINT, DECIMAL(i,j), DATE, TIME (DATETIME), ...

- {column_constraint} is optional constraints on the column such as NULL, NOT NULL, PRIMARY KEY, FOREIGN KEY, UNIQUE, DEFAULT, ...

Example:

For the PROJECTS and TEAMS relations the corresponding tables can be defined as:

- ✓ Projects (PrjId:integer, Name:string, SDate:date, DDate:date, CDate:date)
- ✓ Teams (PrjId:integer, Name:string, Descr:string)

```
CREATE TABLE Projects (  
PrjId INT NOT NULL PRIMARY KEY,  
Name VARCHAR(30) NOT NULL,  
SDate DATE NOT NULL,  
DDate DATE NULL,  
CDate DATE NULL  
)  
CREATE TABLE Teams (  
PrjId INT NOT NULL FOREIGN KEY REFERENCES Projects(PrjId),  
Name VARCHAR(30) NOT NULL,  
Description VARCHAR(100) NULL,  
PRIMARY KEY (PrjId, Name)
```

The primary key constraint in a relation is enforced by using the key word PRIMARY KEY following the key attribute or in case of multiple attributes it can be specified on a separate line as shown in the Teams table above.

The referential integrity constraint in a relational database is implemented by the use of a foreign key. If the referential integrity enforced using a FOREIGN KEY is violated the default SQL statement forces the rejection of the violating tuple. However, by the use of the optional referential triggered actions the designer can attach clauses to the foreign key constraint such as:

- ✓ ON DELETE {CASCADE | NO ACTION | SET DEFAULT | SET NULL}
- ✓ ON UPDATE {CASCADE | NO ACTION | SET DEFAULT | SET NULL}

The default case is NO ACTION, on which the violating action is rejected. CASCADE option ON DELETE deletes all the referencing rows on deletion of a row. SET DEFAULT and SET NULL allow replacing for all the referencing rows the column value by the default value or null value. (Microsoft SQL Server 2000 doesn't support SET DEFAULT and SET NULL)

The ALTER TABLE command allows modification (adding, changing, or dropping) of a column or constraint in a table.

The syntax for the command is:

```
ALTER TABLE <table_name>  
[ALTER COLUMN <column_name> <new_data_type>] |
```

[ADD <column_definition> | <constraint>] |

[DROP <column_name> | <constraint>]

- <table_name> is the name of the table to be altered.

- The ALTER TABLE command takes either of the three optional actions ALTER COLUMN, ADD or DROP. The ALTER COLUMN option modifies an existing column definition, the ADD option adds a new column or constraint and the DROP option drops existing column or constraint.

Example 1:

ALTER TABLE Projects ALTER COLUMN PrjId SMALLINT

ALTER TABLE Teams ALTER COLUMN PrjId SMALLINT

Example 2:

```
CREATE TABLE Projects (  
PrjId SMALLINT NOT NULL,  
Name VARCHAR(30) NOT NULL,  
SDate DATE NOT NULL,  
DDate DATE NULL,  
CDate DATE NULL  
)
```

```
CREATE TABLE Teams (  
TeamId SMALLINT NOT NULL,  
PrjId SMALLINT NOT NULL,  
Name VARCHAR(30) NOT NULL,  
Description VARCHAR(100) NULL,  
)
```

```
ALTER TABLE Projects ADD  
CONSTRAINT [PK_Projects] PRIMARY KEY CLUSTERED (PrjId)
```

```
ALTER TABLE Projects ADD Description VARCAHR(200)
```

```
ALTER TABLE Teams ADD  
CONSTRAINT [PK_Teams] PRIMARY KEY CLUSTERED (TeamId)
```

```
ALTER TABLE Teams ADD  
CONSTRAINT [FK_Teams_Projects] FOREIGN KEY (PrjId)  
REFERENCES Projects (PrjId)
```

The DROP command is used to drop an exiting table, database or schema. The syntax for the command is:

```
DROP TABLE <table_name>
```

```
DROP DATABASE <database_name>
```

DROP SCHEMA <schema_name>

Example:

DROP TABLE Projects

7.2.3 Index Creation and Modification

Indexes are the heart of fast data access. In fact, as the database grows, indexes are the guarantee to fast data access. Data access can be fast without indexes, but only if the table is small. If the table contains thousands or millions of rows, data access has to be done through indexes. Indices in a book, helps to find information about a specific subject without having to read the entire book. The same applies to a database index; it helps to find information about a specific row or rows without having to search through the entire table.

An index for a table is managed by an external table which consists of the search key (index attribute) and a pointer to the location of the data as columns.

Creating indexes is a straightforward process when done with the CREATE INDEX statements.

The basic CREATE INDEX statement is:

```
CREATE [CLUSTERED | NONCLUSTERED] INDEX <index_name>  
ON {<table> | <view> } ( <column> [ ASC | DESC ] [ ,...n ] )
```

Example:

The following statement creates the DueDate nonclustered index on the Projects table:

```
CREATE INDEX DueDate ON Projects(DDate)
```

7.2.3.1 Clustered versus Nonclustered Index

For a clustered index the data is both stored and sorted on the index key; whereas, for a nonclustered index the actual data is not stored in the index. The default index is always nonclustered. One can create a clustered index by specifying it, as in the following example:

Example:

```
CREATE CLUSTERED INDEX PrjId ON Projects(PrjId)
```

NOTE: A table can have only one clustered index. If a primary key constraint is created on a table, a clustered index may be created to support the constraint.

The DROP command is also used with indexes to drop an existing database in a table. The syntax for the command is: DROP INDEX <index_name> [,...n]

Example:

```
DROP INDEX DueDate
```

7.2.3.2 Simple Query Constructs and Syntax

The simplest DML query in the SQL statement is the SELECT-FROM-WHERE statement used for retrieving information from a database. The SQL DML also supports data insertion, modification and deletion through the INSERT INTO, UPDATE and DELETE statements.

✓ The SELECT-FROM-WHERE Statement

The syntax for the SELECT-FROM-WHERE statement which consists of three clauses SELECT, FROM and WHERE as shown below:

```
SELECT <column_list>
```

FROM <table_list>
WHERE <condition>

- <column_list> is the list of column names whose values are retrieved by the query.
 - <table_list> is the list of table names required in the process.
 - <condition> is Boolean expression (conditional expression) that determines the rows to be selected in the query. The expression is build from the logical comparison operators (=, >, <, >=, <= and <>)
- The column list in the SELECT clause can be replaced by an asterisk (*) to retrieve all the columns in the participating tables.

The WHERE clause is an optional clause needed when a condition is to be set for retrieval of rows, if the clause is not used in the statement, all the rows for the selected columns in the specified tables will be retrieved.

Example:

A query to retrieve all the columns for all projects:

SELECT *
FROM Projects

A query to retrieve the name and due date of projects that are not yet completed:

SELECT Name, DDate
FROM Projects
WHERE CDate=NULL

A query to retrieve the projects name and corresponding team names for projects that are not yet completed:

SELECT Projects.Name, Teams.Name
FROM Projects, Teams
WHERE Projects.PrjId=Teams.PrjId AND CDate=NULL

To retrieve all the columns from the team table:

SELECT Projects.Name, Teams.*
FROM Projects, Teams
WHERE Projects.PrjId=Teams.PrjId AND CDate=NULL

In SQL queries it may happen that two participating tables have columns with identical names, to avoid the ambiguity of the columns the name of the table is used together with the column name as shown above. Ambiguity may also arise if a single table is to participate more than once in a query, in such situations an alias may be used for the tables as shown in the following query.

Example:

SELECT p.Name, t.Name
FROM Projects AS p, Teams AS t
WHERE p.PrjId=t.PrjId AND CDate=NULL

The SELECT statement by default results a bag of rows rather than a set of rows (i.e. duplicate rows may exist in the resulting rows). To remove duplicates and have a set of rows as a result one can the DISTINCT key word on the SELECT clause as follows:

SELECT DISTINCT <column_list>
FROM <table_list>
WHERE <condition>

Example:

A query to retrieve employees name, the projects they are participating and due date of the project.

SELECT DISTINCT e.Name, p.Name, p.DDate
FROM Employees AS e, EmpTeams AS et, Teams AS t, Projects AS p
WHERE e.EmpId=et.EmpId AND et.TeamId=t.TeamId AND p.PrjId=t.PrjId

If the SELECT is not DISTINCT the resulting table (view) will include identical set of rows for an employee participating in different teams for same project.

Strings in the WHERE clause can be compared with the use of the comparison operators (=, <, >, <=, >= and <>) and also the LIKE operator that provides the capability to compare strings on the basis of pattern match. The expression is of the form: **S LIKE P**

Where S is the string or the column name to be compared and p is the pattern constructed from two special characters:

- **_** : refers to a match to any one character in S, and
- **%** : refers to zero or more character sequences match in S.

String constants in SQL are enclosed by a single apostrophe. If the string consists of an apostrophe a escape sequence with an apostrophe is used (i.e. two single apostrophes are used to refer to a single apostrophe in a string constant).

The LIKE expression can also be used with the NOT operation as follows: **S NOT LIKE P**

Example:

A query to retrieve employees with a name starting by the letter 'A'.

SELECT *
FROM Employees
WHERE Name='A%'

7.2.4 INSERT, UPDATE and DELETE

7.2.4.1 INSERT

The INSERT statement adds one or more new rows to a table. In a simplified treatment, INSERT has this form: **INSERT INTO <table_name>| <view_name> [(column_list)] data_values** *data_values* are one or more rows to be inserted into the named table or view. *column_list* is a list of column names, separated by commas, that can be used to specify the columns for which data is supplied.

If *column_list* is not specified, all the columns in the table or view receive data. When a *column_list* does not name all the columns in a table or view, a value of NULL (or the default value if a default is defined for the column) is inserted into any column not named in the list. All columns not specified in the column list must either allow null values or have a default assigned.

The data values supplied must match the column list. The number of data values must be the same as the number of columns, and the data type, precision, and scale of each data value must match those of the corresponding column.

There are two ways to specify the data values:

- ✓ VALUES (<value_or_expression> [...n])
- ✓ SELECT <subquery>

The VALUES statement inserts a single row with the column values <value_or_expression> in the columns listed in the INSERT INTO column list. The SELECT subquery is a standard query that results a temporary table and the resulting rows in the table are inserted to the table in the INSERT INTO clause. The columns in the subquery need to match the columns in the columns list.

Example

```
INSERT INTO Projects(PrjId, Name, SDate)
VALUES (1, 'Test Project', '05-25-2006')
INSERT INTO Teams
VALUES (1, 1, 'Programmers Team 1', 'Programmers team for project 2.')
INSERT INTO Teams(TeamId, PrjId, Name)
SELECT TeamId+10, 2, Name FROM Teams WHERE PrjId=1
```

7.2.4.2 UPDATE

The UPDATE statement changes the existing data in a table. The syntax for the UPDATE command is:

```
UPDATE <table_name>| <view_name>
SET <column_name> = <value> [...n]
WHERE <condition>
```

- <value> is new value to be assigned to the column <column_name>
- The WHERE clause specifies the <condition> for selecting the rows to be modified. If the WHERE clause is not included the update will be done for all existing rows in the table

Example

```
UPDATE Teams
SET Description = 'Programmers team for project2'
WHERE TeamId = 11
```

7.2.4.3 DELETE

The DELETE statement removes row(s) from a table. The syntax for the DELETE command is:

```
DELETE FROM <table_name>| <view_name>
WHERE <condition>
```

- ✓ The WHERE clause specifies the <condition> for selecting the rows to be deleted. If the WHERE clause is not included the all existing rows in the table will be deleted unless there is a constraint that protects the deletion of the rows.

Example

```
DELETE Teams
WHERE TeamId=2
```

7.2.5 Nested Subqueries and Complex Queries

The SELECT-FROM-WHERE statement discussed so far is the simplest SQL statement for querying a database. SQL SELECT statements can be combined together to form **Subqueries**. **Subqueries** in a SQL statement are complete form of SELECT-FROM-WHERE statements that are contained in one query.

They can be used in different ways:

- ✓ Subqueries in the WHERE clause to form nested queries,
- ✓ Subqueries in set operations such as UNION, EXCEPT, ..., and
- ✓ Subqueries in the FROM clause as constant tables

7.2.5.1 Nested Queries

SQL SELECT statements can be contained in the WHERE clause of another SQL statement to form **Nested queries**. The SELECT statement that contains the nested query is said to be the **outer query**. Subqueries in a nested SQL statement can produce scalar value (constant) or table.

Subqueries resulting scalar value can be used in comparison expression of the WHERE clause similar to constant or column value comparisons. For subqueries that result table special operators are used in the test expression such as the operator IN that is used to test the existence of a scalar value in the resulting table.

Example:

Considering the following relations,

- ✓ Employees(EmpId, Name, BDate, SubCity, Kebele, Phone, Salary)
- ✓ Teams(TeamId, PrjId, Name, Descr)
- ✓ EmpTeams(EmpId, TeamId)
- ✓ Projects(PrjId, Name, SDate, DDate, CDate, CustId)
- ✓ Customers(CustId, Name, Address)

Write a query to retrieve all the projects that are owned by the customer 'XYZ'. (Assume name of a customer is unique)

```
SELECT Name, SDate, DDate, CDate  
FROM Projects  
WHERE CustId = (SELECT CustId  
FROM Customers  
WHERE Name='XYZ')
```

Alternative for the above query is;

```
SELECT p.Name, SDate, DDate, CDate  
FROM Projects AS p, Customers AS c  
WHERE p.CustId = c.CustId AND c.Name='XYZ'
```

Write a query to retrieve employees name and phone that are participating on projects that are owned by the customer 'XYZ'.

```
SELECT Name, Phone  
FROM Employees
```

**WHERE EmpId IN (SELECT EmpId
FROM EmpTeams AS et, Teams AS t
WHERE et.TeamId= t.TeamId AND
PrjId IN (SELECT PrjId
FROM Projects AS p, Customers AS c
WHERE p.CustId=c.CustId
AND c.Name='XYZ'))**